| TERM | DEFINITION |
|---|---|
| *prefix* of $s$ | A string obtained by removing zero or more trailing symbols of string $s$; e.g., **ban** is a prefix of **banana**. |
| *suffix* of $s$ | A string formed by deleting zero or more of the leading symbols of $s$; e.g., **nana** is a suffix of **banana**. |
| *substring* of $s$ | A string obtained by deleting a prefix and a suffix from $s$; e.g., **nan** is a substring of **banana**. Every prefix and every suffix of $s$ is a substring of $s$, but not every substring of $s$ is a prefix or a suffix of $s$. For every string $s$, both $s$ and $\epsilon$ are prefixes, suffixes, and substrings of $s$. |
| *proper* prefix, suffix, or substring of $s$ | Any nonempty string $x$ that is, respectively, a prefix, suffix, or substring of $s$ such that $s \neq x$. |
| *subsequence* of $s$ | Any string formed by deleting zero or more not necessarily contiguous symbols from $s$; e.g., **baaa** is a subsequence of **banana**. |

**Fig. 3.7.** Terms for parts of a string.

### Operations on Languages

There are several important operations that can be applied to languages. For lexical analysis, we are interested primarily in union, concatenation, and closure, which are defined in Fig. 3.8. We can also generalize the "exponentiation" operator to languages by defining $L^0$ to be $\{\epsilon\}$, and $L^i$ to be $L^{i-1}L$. Thus, $L^i$ is $L$ concatenated with itself $i-1$ times.

**Example 3.2.** Let $L$ be the set {A, B, . . . , Z, a, b, . . . , z} and $D$ the set {0, 1, . . . , 9}. We can think of $L$ and $D$ in two ways. We can think of $L$ as the alphabet consisting of the set of upper and lower case letters, and $D$ as the alphabet consisting of the set of the ten decimal digits. Alternatively, since a symbol can be regarded as a string of length one, the sets $L$ and $D$ are each finite languages. Here are some examples of new languages created from $L$ and $D$ by applying the operators defined in Fig. 3.8.

1. $L \cup D$ is the set of letters and digits.
2. $LD$ is the set of strings consisting of a letter followed by a digit.
3. $L^4$ is the set of all four-letter strings.
4. $L^*$ is the set of all strings of letters, including $\epsilon$, the empty string.
5. $L(L \cup D)^*$ is the set of all strings of letters and digits beginning with a letter.
6. $D^+$ is the set of all strings of one or more digits. □

| OPERATION | DEFINITION |
|---|---|
| *union* of $L$ and $M$ written $L \cup M$ | $L \cup M = \{ s \mid s \text{ is in } L \text{ or } s \text{ is in } M \}$ |
| *concatenation* of $L$ and $M$ written $LM$ | $LM = \{ st \mid s \text{ is in } L \text{ and } t \text{ is in } M \}$ |
| *Kleene closure* of $L$ written $L^*$ | $L^* = \bigcup_{i=0}^{\infty} L^i$ <br> $L^*$ denotes "zero or more concatenations of" $L$. |
| *positive closure* of $L$ written $L^+$ | $L^+ = \bigcup_{i=1}^{\infty} L^i$ <br> $L^+$ denotes "one or more concatenations of" $L$. |

**Fig. 3.8.** Definitions of operations on languages.

### Regular Expressions

In Pascal, an identifier is a letter followed by zero or more letters or digits; that is, an identifier is a member of the set defined in part (5) of Example 3.2. In this section, we present a notation, called regular expressions, that allows us to define precisely sets such as this. With this notation, we might define Pascal identifiers as

**letter ( letter | digit ) ***

The vertical bar here means "or," the parentheses are used to group subexpressions, the star means "zero or more instances of" the parenthesized expression, and the juxtaposition of **letter** with the remainder of the expression means concatenation.

A regular expression is built up out of simpler regular expressions using a set of defining rules. Each regular expression $r$ denotes a language $L(r)$. The defining rules specify how $L(r)$ is formed by combining in various ways the languages denoted by the subexpressions of $r$.

Here are the rules that define the *regular expressions over alphabet* $\Sigma$. Associated with each rule is a specification of the language denoted by the regular expression being defined.

1. $\epsilon$ is a regular expression that denotes $\{\epsilon\}$, that is, the set containing the empty string.

2. If $a$ is a symbol in $\Sigma$, then $a$ is a regular expression that denotes $\{a\}$, i.e., the set containing the string $a$. Although we use the same notation for all three, technically, the regular expression $a$ is different from the string $a$ or the symbol $a$. It will be clear from the context whether we are talking about $a$ as a regular expression, string, or symbol.

3. Suppose $r$ and $s$ are regular expressions denoting the languages $L(r)$ and $L(s)$. Then,

   a) $(r)|(s)$ is a regular expression denoting $L(r) \cup L(s)$.
   b) $(r)(s)$ is a regular expression denoting $L(r)L(s)$.
   c) $(r)^*$ is a regular expression denoting $(L(r))^*$.
   d) $(r)$ is a regular expression denoting $L(r)$.[2]

A language denoted by a regular expression is said to be a *regular set*.

The specification of a regular expression is an example of a recursive definition. Rules (1) and (2) form the basis of the definition; we use the term *basic symbol* to refer to $\epsilon$ or a symbol in $\Sigma$ appearing in a regular expression. Rule (3) provides the inductive step.

Unnecessary parentheses can be avoided in regular expressions if we adopt the conventions that:

1. the unary operator * has the highest precedence and is left associative,
2. concatenation has the second highest precedence and is left associative,
3. | has the lowest precedence and is left associative.

Under these conventions, $(a)|((b)^*(c))$ is equivalent to $a|b^*c$. Both expressions denote the set of strings that are either a single $a$ or zero or more $b$'s followed by one $c$.

**Example 3.3.** Let $\Sigma = \{a, b\}$.

1. The regular expression $a|b$ denotes the set $\{a, b\}$.
2. The regular expression $(a|b)(a|b)$ denotes $\{aa, ab, ba, bb\}$, the set of all strings of $a$'s and $b$'s of length two. Another regular expression for this same set is $aa \mid ab \mid ba \mid bb$.
3. The regular expression $a^*$ denotes the set of all strings of zero or more $a$'s, i.e., $\{\epsilon, a, aa, aaa, \cdots\}$.
4. The regular expression $(a|b)^*$ denotes the set of all strings containing zero or more instances of an $a$ or $b$, that is, the set of all strings of $a$'s and $b$'s. Another regular expression for this set is $(a^*b^*)^*$.
5. The regular expression $a \mid a^*b$ denotes the set containing the string $a$ and all strings consisting of zero or more $a$'s followed by a $b$. □

If two regular expressions $r$ and $s$ denote the same language, we say $r$ and $s$ are *equivalent* and write $r = s$. For example, $(a|b) = (b|a)$.

There are a number of algebraic laws obeyed by regular expressions and these can be used to manipulate regular expressions into equivalent forms. Figure 3.9 shows some algebraic laws that hold for regular expressions $r$, $s$, and $t$.

[2] This rule says that extra pairs of parentheses may be placed around regular expressions if we desire.

| AXIOM | DESCRIPTION |
|---|---|
| $r\|s = s\|r$ | $\|$ is commutative |
| $r\|(s\|t) = (r\|s)\|t$ | $\|$ is associative |
| $(rs)t = r(st)$ | concatenation is associative |
| $r(s\|t) = rs\|rt$<br>$(s\|t)r = sr\|tr$ | concatenation distributes over $\|$ |
| $\epsilon r = r$<br>$r\epsilon = r$ | $\epsilon$ is the identity element for concatenation |
| $r^* = (r\|\epsilon)^*$ | relation between * and $\epsilon$ |
| $r^{**} = r^*$ | * is idempotent |

**Fig. 3.9.** Algebraic properties of regular expressions.

### Regular Definitions

For notational convenience, we may wish to give names to regular expressions and to define regular expressions using these names as if they were symbols. If $\Sigma$ is an alphabet of basic symbols, then a *regular definition* is a sequence of definitions of the form

$$\begin{aligned} d_1 &\rightarrow r_1 \\ d_2 &\rightarrow r_2 \\ &\cdots \\ d_n &\rightarrow r_n \end{aligned}$$

where each $d_i$ is a distinct name, and each $r_i$ is a regular expression over the symbols in $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$, i.e., the basic symbols and the previously defined names. By restricting each $r_i$ to symbols of $\Sigma$ and the previously defined names, we can construct a regular expression over $\Sigma$ for any $r_i$ by repeatedly replacing regular-expression names by the expressions they denote. If $r_i$ used $d_j$ for some $j \geq i$, then $r_i$ might be recursively defined, and this substitution process would not terminate.

To distinguish names from symbols, we print the names in regular definitions in boldface.

**Example 3.4.** As we have stated, the set of Pascal identifiers is the set of strings of letters and digits beginning with a letter. Here is a regular definition for this set.

**letter** → `A` | `B` | · · · | `Z` | `a` | `b` | · · · | `z`
**digit** → `0` | `1` | · · · | `9`
**id** → **letter** ( **letter** | **digit** )* □

**Example 3.5.** Unsigned numbers in Pascal are strings such as `5280`, `39.37`,

`6.336E4`, or `1.894E-4`. The following regular definition provides a precise specification for this class of strings:

$$\begin{aligned}
\textbf{digit} &\rightarrow \texttt{0} \mid \texttt{1} \mid \cdots \mid \texttt{9} \\
\textbf{digits} &\rightarrow \textbf{digit digit}^* \\
\textbf{optional_fraction} &\rightarrow \texttt{.}\ \textbf{digits} \mid \epsilon \\
\textbf{optional_exponent} &\rightarrow (\ \texttt{E}\ (\ \texttt{+} \mid \texttt{-} \mid \epsilon\ )\ \textbf{digits}\ ) \mid \epsilon \\
\textbf{num} &\rightarrow \textbf{digits optional_fraction optional_exponent}
\end{aligned}$$

This definition says that an **optional_fraction** is either a decimal point followed by one or more digits, or it is missing (the empty string). An **optional_exponent**, if it is not missing, is an `E` followed by an optional `+` or `-` sign, followed by one or more digits. Note that at least one digit must follow the period, so **num** does not match `1.` but it does match `1.0`. □

### Notational Shorthands

Certain constructs occur so frequently in regular expressions that it is convenient to introduce notational shorthands for them.

1. *One or more instances.* The unary postfix operator $^+$ means "one or more instances of." If $r$ is a regular expression that denotes the language $L(r)$, then $(r)^+$ is a regular expression that denotes the language $(L(r))^+$. Thus, the regular expression $a^+$ denotes the set of all strings of one or more $a$'s. The operator $^+$ has the same precedence and associativity as the operator *. The two algebraic identities $r^* = r^+ \mid \epsilon$ and $r^+ = rr^*$ relate the Kleene and positive closure operators.

2. *Zero or one instance.* The unary postfix operator ? means "zero or one instance of." The notation $r?$ is a shorthand for $r \mid \epsilon$. If $r$ is a regular expression, then $(r)?$ is a regular expression that denotes the language $L(r) \cup \{\epsilon\}$. For example, using the $^+$ and ? operators, we can rewrite the regular definition for **num** in Example 3.5 as

$$\begin{aligned}
\textbf{digit} &\rightarrow \texttt{0} \mid \texttt{1} \mid \cdots \mid \texttt{9} \\
\textbf{digits} &\rightarrow \textbf{digit}^+ \\
\textbf{optional_fraction} &\rightarrow (\ \texttt{.}\ \textbf{digits}\ )? \\
\textbf{optional_exponent} &\rightarrow (\ \texttt{E}\ (\ \texttt{+} \mid \texttt{-}\ )?\ \textbf{digits}\ )? \\
\textbf{num} &\rightarrow \textbf{digits optional_fraction optional_exponent}
\end{aligned}$$

3. *Character classes.* The notation `[abc]` where `a`, `b`, and `c` are alphabet symbols denotes the regular expression `a | b | c`. An abbreviated character class such as `[a-z]` denotes the regular expression `a | b | ··· | z`. Using character classes, we can describe identifiers as being strings generated by the regular expression

```
[A-Za-z][A-Za-z0-9]*
```

### Nonregular Sets

Some languages cannot be described by any regular expression. To illustrate the limits of the descriptive power of regular expressions, here we give examples of programming language constructs that cannot be described by regular expressions. Proofs of these assertions can be found in the references.

Regular expressions cannot be used to describe balanced or nested constructs. For example, the set of all strings of balanced parentheses cannot be described by a regular expression. On the other hand, this set can be specified by a context-free grammar.

Repeating strings cannot be described by regular expressions. The set

$$\{wcw \mid w \text{ is a string of } a\text{'s and } b\text{'s}\}$$

cannot be denoted by any regular expression, nor can it be described by a context-free grammar.

Regular expressions can be used to denote only a fixed number of repetitions or an unspecified number of repetitions of a given construct. Two arbitrary numbers cannot be compared to see whether they are the same. Thus, we cannot describe Hollerith strings of the form $nHa_1a_2 \cdots a_n$ from early versions of Fortran with a regular expression, because the number of characters following H must match the decimal number before H.

## 3.4 RECOGNITION OF TOKENS

In the previous section, we considered the problem of how to specify tokens. In this section, we address the question of how to recognize them. Throughout this section, we use the language generated by the following grammar as a running example.

**Example 3.6.** Consider the following grammar fragment:

*stmt* → **if** *expr* **then** *stmt*
| **if** *expr* **then** *stmt* **else** *stmt*
| $\epsilon$
*expr* → *term* **relop** *term*
| *term*
*term* → **id**
| **num**

where the terminals **if**, **then**, **else**, **relop**, **id**, and **num** generate sets of strings given by the following regular definitions:

**if** → `if`
**then** → `then`
**else** → `else`
**relop** → `<` | `<=` | `=` | `<>` | `>` | `>=`
**id** → **letter** ( **letter** | **digit** )*
**num** → **digit**$^+$ ( `.` **digit**$^+$ )? ( `E` ( `+`|`-` )? **digit**$^+$ )?

where **letter** and **digit** are as defined previously.

For this language fragment the lexical analyzer will recognize the keywords `if`, `then`, `else`, as well as the lexemes denoted by **relop**, **id**, and **num**. To simplify matters, we assume keywords are reserved; that is, they cannot be used as identifiers. As in Example 3.5, **num** represents the unsigned integer and real numbers of Pascal.

In addition, we assume lexemes are separated by white space, consisting of nonnull sequences of blanks, tabs, and newlines. Our lexical analyzer will strip out white space. It will do so by comparing a string against the regular definition **ws**, below.

$$\textbf{delim} \rightarrow \textbf{blank} \mid \textbf{tab} \mid \textbf{newline}$$
$$\textbf{ws} \rightarrow \textbf{delim}^{+}$$

If a match for **ws** is found, the lexical analyzer does not return a token to the parser. Rather, it proceeds to find a token following the white space and returns that to the parser.

Our goal is to construct a lexical analyzer that will isolate the lexeme for the next token in the input buffer and produce as output a pair consisting of the appropriate token and attribute-value, using the translation table given in Fig. 3.10. The attribute-values for the relational operators are given by the symbolic constants **LT**, **LE**, **EQ**, **NE**, **GT**, **GE**. □

| REGULAR EXPRESSION | TOKEN | ATTRIBUTE-VALUE |
|---|---|---|
| **ws** | - | - |
| `if` | **if** | - |
| `then` | **then** | - |
| `else` | **else** | - |
| **id** | **id** | pointer to table entry |
| **num** | **num** | pointer to table entry |
| < | **relop** | **LT** |
| <= | **relop** | **LE** |
| = | **relop** | **EQ** |
| <> | **relop** | **NE** |
| > | **relop** | **GT** |
| >= | **relop** | **GE** |

**Fig. 3.10.** Regular-expression patterns for tokens.

### Transition Diagrams

As an intermediate step in the construction of a lexical analyzer, we first produce a stylized flowchart, called a *transition diagram*. Transition diagrams

depict the actions that take place when a lexical analyzer is called by the parser to get the next token, as suggested by Fig. 3.1. Suppose the input buffer is as in Fig. 3.3 and the lexeme-beginning pointer points to the character following the last lexeme found. We use a transition diagram to keep track of information about characters that are seen as the forward pointer scans the input. We do so by moving from position to position in the diagram as characters are read.

Positions in a transition diagram are drawn as circles and are called *states*. The states are connected by arrows, called *edges*. Edges leaving state $s$ have labels indicating the input characters that can next appear after the transition diagram has reached state $s$. The label **other** refers to any character that is not indicated by any of the other edges leaving $s$.

We assume the transition diagrams of this section are *deterministic*; that is, no symbol can match the labels of two edges leaving one state. Starting in Section 3.5, we shall relax this condition, making life much simpler for the designer of the lexical analyzer and, with proper tools, no harder for the implementor.

One state is labeled the *start* state; it is the initial state of the transition diagram where control resides when we begin to recognize a token. Certain states may have actions that are executed when the flow of control reaches that state. On entering a state we read the next input character. If there is an edge from the current state whose label matches this input character, we then go to the state pointed to by the edge. Otherwise, we indicate failure.

Figure 3.11 shows a transition diagram for the patterns >= and >. The transition diagram works as follows. Its start state is state 0. In state 0, we read the next input character. The edge labeled > from state 0 is to be followed to state 6 if this input character is >. Otherwise, we have failed to recognize either > or >=.

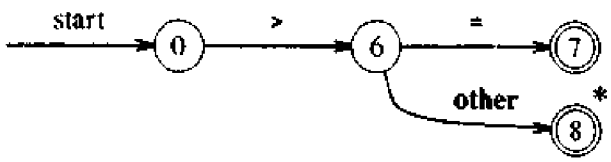



**Fig. 3.11.** Transition diagram for >=.

On reaching state 6 we read the next input character. The edge labeled = from state 6 is to be followed to state 7 if this input character is an =. Otherwise, the edge labeled **other** indicates that we are to go to state 8. The double circle on state 7 indicates that it is an accepting state, a state in which the token >= has been found.

Notice that the character > and another extra character are read as we follow the sequence of edges from the start state to the accepting state 8. Since the extra character is not a part of the relational operator >, we must retract

the forward pointer one character. We use a * to indicate states on which this input retraction must take place.

In general, there may be several transition diagrams, each specifying a group of tokens. If failure occurs while we are following one transition diagram, then we retract the forward pointer to where it was in the start state of this diagram, and activate the next transition diagram. Since the lexeme-beginning and forward pointers marked the same position in the start state of the diagram, the forward pointer is retracted to the position marked by the lexeme-beginning pointer. If failure occurs in all transition diagrams, then a lexical error has been detected and we invoke an error-recovery routine.

**Example 3.7.** A transition diagram for the token **relop** is shown in Fig. 3.12. Notice that Fig. 3.11 is a part of this more complex transition diagram. □

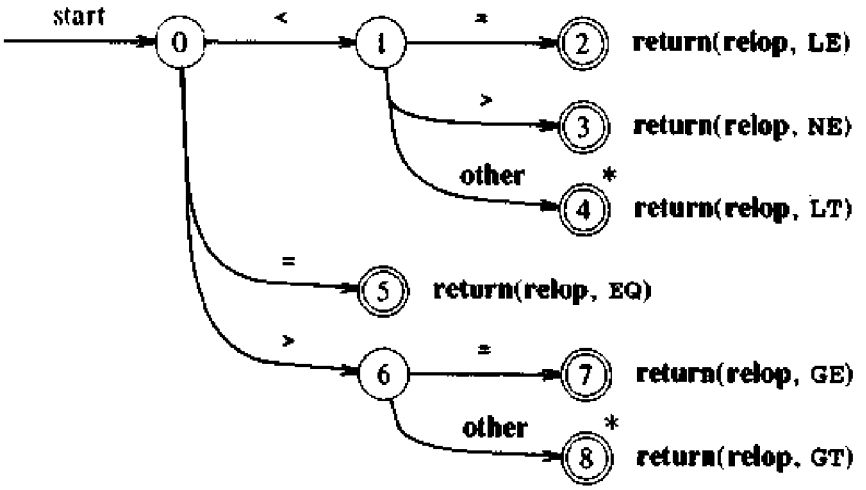



**Fig. 3.12.** Transition diagram for relational operators.

**Example 3.8.** Since keywords are sequences of letters, they are exceptions to the rule that a sequence of letters and digits starting with a letter is an identifier. Rather than encode the exceptions into a transition diagram, a useful trick is to treat keywords as special identifiers, as in Section 2.7. When the accepting state in Fig. 3.13 is reached, we execute some code to determine if the lexeme leading to the accepting state is a keyword or an identifier.

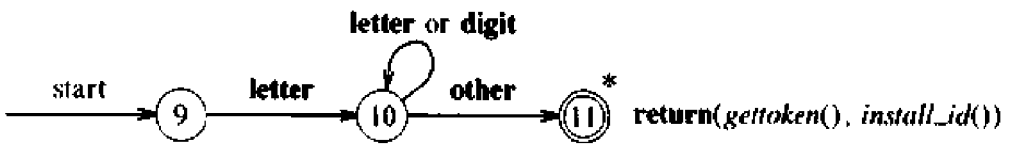



**Fig. 3.13.** Transition diagram for identifiers and keywords.

A simple technique for separating keywords from identifiers is to initialize appropriately the symbol table in which information about identifiers is saved. For the tokens of Fig. 3.10 we need to enter the strings `if`, `then`, and `else` into the symbol table before any characters in the input are seen. We also make a note in the symbol table of the token to be returned when one of these strings is recognized. The return statement next to the accepting state in Fig. 3.13 uses *gettoken*() and *install_id*() to obtain the token and attribute-value, respectively, to be returned. The procedure *install_id*() has access to the buffer, where the identifier lexeme has been located. The symbol table is examined and if the lexeme is found there marked as a keyword, *install_id*() returns 0. If the lexeme is found and is a program variable, *install_id*() returns a pointer to the symbol table entry. If the lexeme is not found in the symbol table, it is installed as a variable and a pointer to the newly created entry is returned.

The procedure *gettoken*() similarly looks for the lexeme in the symbol table. If the lexeme is a keyword, the corresponding token is returned; otherwise, the token **id** is returned.

Note that the transition diagram does not change if additional keywords are to be recognized; we simply initialize the symbol table with the strings and tokens of the additional keywords. □

The technique of placing keywords in the symbol table is almost essential if the lexical analyzer is coded by hand. Without doing so, the number of states in a lexical analyzer for a typical programming language is several hundred, while using the trick, fewer than a hundred states will probably suffice.

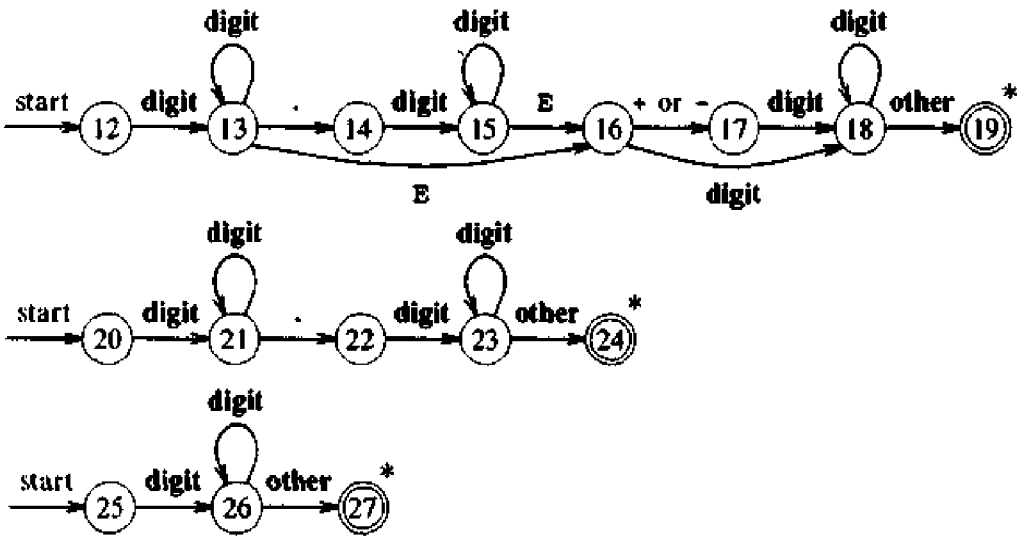



Fig. 3.14. Transition diagrams for unsigned numbers in Pascal.

**Example 3.9.** A number of issues arise when we construct a recognizer for unsigned numbers given by the regular definition

**num** → **digit**$^+$ ( . **digit**$^+$ )? ( E( + | - )? **digit**$^+$ )?

Note that the definition is of the form **digits fraction**? **exponent**? in which **fraction** and **exponent** are optional.

The lexeme for a given token must be the longest possible. For example, the lexical analyzer must not stop after seeing `12` or even `12.3` when the input is `12.3E4`. Starting at states 25, 20, and 12 in Fig. 3.14, accepting states will be reached after `12`, `12.3`, and `12.3E4` are seen, respectively, provided `12.3E4` is followed by a non-digit in the input. The transition diagrams with start states 25, 20, and 12 are for **digits**, **digits fraction**, and **digits fraction**? **exponent**, respectively, so the start states must be tried in the reverse order 12, 20, 25.

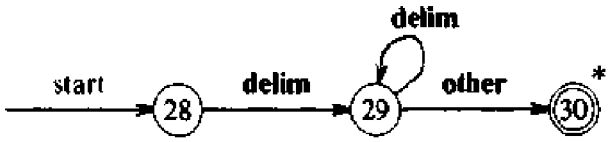
The action when any of the accepting states 19, 24, or 27 is reached is to call a procedure *install_num* that enters the lexeme into a table of numbers and returns a pointer to the created entry. The lexical analyzer returns the token **num** with this pointer as the lexical value. □

Information about the language that is not in the regular definitions of the tokens can be used to pinpoint errors in the input. For example, on input `1. < x`, we fail in states 14 and 22 in Fig. 3.14 with next input character <. Rather than returning the number 1, we may wish to report an error and continue as if the input were `1.0 < x`. Such knowledge can also be used to simplify the transition diagrams, because error-handling may be used to recover from some situations that would otherwise lead to failure.

There are several ways in which the redundant matching in the transition diagrams of Fig. 3.14 can be avoided. One approach is to rewrite the transition diagrams by combining them into one, a nontrivial task in general. Another is to change the response to failure during the process of following a diagram. An approach explored later in this chapter allows us to pass through several accepting states; we revert back to the last accepting state that we passed through when failure occurs.

**Example 3.10.** A sequence of transition diagrams for all tokens of Example 3.6 is obtained if we put together the transition diagrams of Fig. 3.12, 3.13, and 3.14. Lower-numbered start states are to be attempted before higher numbered states.

The only remaining issue concerns white space. The treatment of **ws**, representing white space, is different from that of the patterns discussed above because nothing is returned to the parser when white space is found in the input. A transition diagram recognizing **ws** by itself is




Nothing is returned when the accepting state is reached; we merely go back to the start state of the first transition diagram to look for another pattern.

Whenever possible, it is better to look for frequently occurring tokens before less frequently occurring ones, because a transition diagram is reached only after we fail on all earlier diagrams. Since white space is expected to occur frequently, putting the transition diagram for white space near the beginning should be an improvement over testing for white space at the end. □

### Implementing a Transition Diagram

A sequence of transition diagrams can be converted into a program to look for the tokens specified by the diagrams. We adopt a systematic approach that works for all transition diagrams and constructs programs whose size is proportional to the number of states and edges in the diagrams.

Each state gets a segment of code. If there are edges leaving a state, then its code reads a character and selects an edge to follow, if possible. A function `nextchar()` is used to read the next character from the input buffer, advance the forward pointer at each call, and return the character read.[3] If there is an edge labeled by the character read, or labeled by a character class containing the character read, then control is transferred to the code for the state pointed to by that edge. If there is no such edge, and the current state is not one that indicates a token has been found, then a routine `fail()` is invoked to retract the forward pointer to the position of the beginning pointer and to initiate a search for a token specified by the next transition diagram. If there are no other transition diagrams to try, `fail()` calls an error-recovery routine.

To return tokens we use a global variable `lexical_value`, which is assigned the pointers returned by functions `install_id()` and `install_num()` when an identifier or number, respectively, is found. The token class is returned by the main procedure of the lexical analyzer, called `nexttoken()`.

We use a case statement to find the start state of the next transition diagram. In the C implementation in Fig. 3.15, two variables `state` and `start` keep track of the present state and the starting state of the current transition diagram. The state numbers in the code are for the transition diagrams of Figures 3.12 – 3.14.

Edges in transition diagrams are traced by repeatedly selecting the code fragment for a state and executing the code fragment to determine the next state as shown in Fig. 3.16. We show the code for state 0, as modified in Example 3.10 to handle white space, and the code for two of the transition diagrams from Fig. 3.13 and 3.14. Note that the C construct

```
while(1) stmt
```

repeats *stmt* "forever," i.e., until a `return` occurs.

[3] A more efficient implementation would use an in-line macro in place of the function `nextchar()`.

```
int state = 0, start = 0;
int lexical_value;
    /* to "return" second component of token */

int fail()
{
    forward = token_beginning;
    switch (start) {
        case 0:   start = 9; break;
        case 9:   start = 12; break;
        case 12:  start = 20; break;
        case 20:  start = 25; break;
        case 25:  recover(); break;
        default:   /* compiler error */
    }
    return start;
}
```

**Fig. 3.15.** C code to find next start state.

Since C does not allow both a token and an attribute-value to be returned, `install_id()` and `install_num()` appropriately set some global variable to the attribute-value corresponding to the table entry for the **id** or **num** in question.

If the implementation language does not have a case statement, we can create an array for each state, indexed by characters. If *state* [ is such an array, then *state* [[*c*]] is a pointer to a piece of code that must be executed whenever the lookahead character is *c*. This code would normally end with a goto to code for the next state. The array for state *s* is referred to as the indirect transfer table for *s*.

## 3.5 A LANGUAGE FOR SPECIFYING LEXICAL ANALYZERS

Several tools have been built for constructing lexical analyzers from special-purpose notations based on regular expressions. We have already seen the use of regular expressions for specifying token patterns. Before we consider algorithms for compiling regular expressions into pattern-matching programs, we give an example of a tool that might use such an algorithm.

In this section, we describe a particular tool, called Lex, that has been widely used to specify lexical analyzers for a variety of languages. We refer to the tool as the *Lex compiler*, and to its input specification as the *Lex language*. Discussion of an existing tool will allow us to show how the specification of patterns using regular expressions can be combined with actions, e.g., making entries into a symbol table, that a lexical analyzer may be required to perform. Lex-like specifications can be used even if a Lex

```
token nexttoken()
{   while(1) {
        switch (state) {
        case 0:   c = nextchar();
           /* c is lookahead character */
           if (c==blank || c==tab || c==newline) {
              state = 0;
              lexeme_beginning++;
                 /* advance beginning of lexeme */
           }
           else if (c == '<') state = 1;
           else if (c == '=') state = 5;
           else if (c == '>') state = 6;
           else state = fail();
           break;

           ... /* cases 1-8 here */

        case 9:   c = nextchar();
           if (isletter(c)) state = 10;
           else state = fail();
           break;
        case 10:  c = nextchar();
           if (isletter(c)) state = 10;
           else if (isdigit(c)) state = 10;
           else state = 11;
           break;
        case 11:  retract(1); install_id();
           return ( gettoken() );

           ... /* cases 12-24 here */

        case 25:  c = nextchar();
           if  (isdigit(c)) state = 26;
           else state = fail();
           break;
        case 26:  c = nextchar();
           if (isdigit(c)) state = 26;
           else state = 27;
           break;
        case 27:  retract(1); install_num();
           return ( NUM );
        }
     }
}
```

**Fig. 3.16.** C code for lexical analyzer.

compiler is not available; the specifications can be manually transcribed into a working program using the transition diagram techniques of the previous section.

Lex is generally used in the manner depicted in Fig. 3.17. First, a specification of a lexical analyzer is prepared by creating a program `lex.l` in the Lex language. Then, `lex.l` is run through the Lex compiler to produce a C program `lex.yy.c`. The program `lex.yy.c` consists of a tabular representation of a transition diagram constructed from the regular expressions of `lex.l`, together with a standard routine that uses the table to recognize lexemes. The actions associated with regular expressions in `lex.l` are pieces of C code and are carried over directly to `lex.yy.c`. Finally, `lex.yy.c` is run through the C compiler to produce an object program `a.out`, which is the lexical analyzer that transforms an input stream into a sequence of tokens.

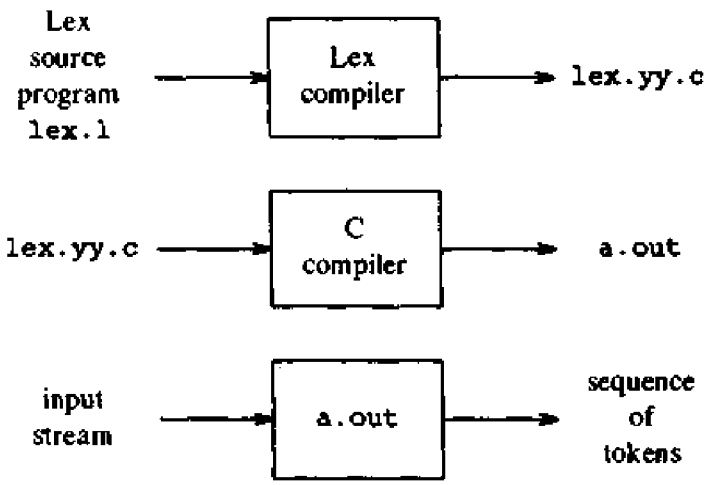



**Fig. 3.17.** Creating a lexical analyzer with Lex.

### Lex Specifications

A Lex program consists of three parts:

declarations
%%
translation rules
%%
auxiliary procedures

The declarations section includes declarations of variables, manifest constants, and regular definitions. (A manifest constant is an identifier that is declared to represent a constant.) The regular definitions are statements similar to those given in Section 3.3 and are used as components of the regular expressions appearing in the translation rules.

The translation rules of a Lex program are statements of the form

$$\begin{array}{ll} p_1 & \{ action_1 \} \\ p_2 & \{ action_2 \} \\ \cdots & \cdots \\ p_n & \{ action_n \} \end{array}$$

where each $p_i$ is a regular expression and each $action_i$ is a program fragment describing what action the lexical analyzer should take when pattern $p_i$ matches a lexeme. In Lex, the actions are written in C; in general, however, they can be in any implementation language.

The third section holds whatever auxiliary procedures are needed by the actions. Alternatively, these procedures can be compiled separately and loaded with the lexical analyzer.

A lexical analyzer created by Lex behaves in concert with a parser in the following manner. When activated by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it has found the longest prefix of the input that is matched by one of the regular expressions $p_i$. Then, it executes $action_i$. Typically, $action_i$ will return control to the parser. However, if it does not, then the lexical analyzer proceeds to find more lexemes, until an action causes control to return to the parser. The repeated search for lexemes until an explicit return allows the lexical analyzer to process white space and comments conveniently.

The lexical analyzer returns a single quantity, the token, to the parser. To pass an attribute value with information about the lexeme, we can set a global variable called `yylval`.

**Example 3.11.** Figure 3.18 is a Lex program that recognizes the tokens of Fig. 3.10 and returns the token found. A few observations about the code will introduce us to many of the important features of Lex.

In the declarations section, we see (a place for) the declaration of certain manifest constants used by the translation rules.[4] These declarations are surrounded by the special brackets `%{` and `%}`. Anything appearing between these brackets is copied directly into the lexical analyzer `lex.yy.c`, and is not treated as part of the regular definitions or the translation rules. Exactly the same treatment is accorded the auxiliary procedures in the third section. In Fig. 3.18, there are two procedures, `install_id` and `install_num`, that are used by the translation rules; these procedures will be copied into `lex.yy.c` verbatim.

Also included in the definitions section are some regular definitions. Each such definition consists of a name and a regular expression denoted by that name. For example, the first name defined is `delim`; it stands for the

[4] It is common for the program `lex.yy.c` to be used as a subroutine of a parser generated by Yacc, a parser generator to be discussed in Chapter 4. In this case, the declaration of the manifest constants would be provided by the parser, when it is compiled with the program `lex.yy.c`.

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}

/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+\-]?{digit}+)?

%%

{ws}        {/* no action and no return */}
if          {return(IF);}
then        {return(THEN);}
else        {return(ELSE);}
{id}        {yylval = install_id(); return(ID);}
{number}    {yylval = install_num(); return(NUMBER);}
"<"         {yylval = LT; return(RELOP);}
"<="        {yylval = LE; return(RELOP);}
"="         {yylval = EQ; return(RELOP);}
"<>"        {yylval = NE; return(RELOP);}
">"         {yylval = GT; return(RELOP);}
">="        {yylval = GE; return(RELOP);}

%%

install_id() {
    /* procedure to install the lexeme, whose
    first character is pointed to by yytext and
    whose length is yyleng, into the symbol table
    and return a pointer thereto */
}

install_num() {
    /* similar procedure to install a lexeme that
    is a number */
}
```

**Fig. 3.18.** Lex program for the tokens of Fig. 3.10.

character class `[ \t\n]`, that is, any of the three symbols blank, tab (represented by `\t`), or newline (represented by `\n`). The second definition is of white space, denoted by the name `ws`. White space is any sequence of one or more delimiter characters. Notice that the word `delim` must be surrounded by braces in Lex, to distinguish it from the pattern consisting of the five letters `delim`.

In the definition of `letter`, we see the use of a character class. The shorthand `[A-Za-z]` means any of the capital letters `A` through `Z` or the lowercase letters `a` through `z`. The fifth definition, of `id`, uses parentheses, which are metasymbols in Lex, with their natural meaning as groupers. Similarly, the vertical bar is a Lex metasymbol representing union.

In the last regular definition, of `number`, we observe a few more details. We see `?` used as a metasymbol, with its customary meaning of "zero or one occurrences of." We also note the backslash used as an escape, to let a character that is a Lex metasymbol have its natural meaning. In particular, the decimal point in the definition of `number` is expressed by `\.` because a dot by itself represents the character class of all characters except the newline, in Lex as in many UNIX system programs that deal with regular expressions. In the character class `[+\-]`, we placed a backslash before the minus sign because the minus sign standing for itself could be confused with its use to denote a range, as in `[A-Z]`.[5]

There is another way to cause characters to have their natural meaning, even if they are metasymbols of Lex: surround them with quotes. We have shown an example of this convention in the translation rules section, where the six relational operators are surrounded by quotes.[6]

Now, let us consider the translation rules in the section following the first `%%`. The first rule says that if we see `ws`, that is, any maximal sequence of blanks, tabs, and newlines, we take no action. In particular, we do not return to the parser. Recall that the structure of the lexical analyzer is such that it keeps trying to recognize tokens, until the action associated with one found causes a return.

The second rule says that if the letters `if` are seen, return the token `IF`, which is a manifest constant representing some integer understood by the parser to be the token `if`. The next two rules handle keywords `then` and `else` similarly.

In the rule for `id`, we see two statements in the associated action. First, the variable `yylval` is set to the value returned by procedure `install_id`; the definition of that procedure is in the third section. `yylval` is a variable

[5] Actually, Lex handles the character class `[+-]` correctly without the backslash, because the minus sign appearing at the end cannot represent a range.

[6] We did so because `<` and `>` are Lex metasymbols; they surround the names of "states," enabling Lex to change state when encountering certain tokens, such as comments or quoted strings, that must be treated differently from the usual text. There is no need to surround the equal sign by quotes, but neither is it forbidden.

whose definition appears in the Lex output `lex.yy.c`, and which is also available to the parser. The purpose of `yylval` is to hold the lexical value returned, since the second statement of the action, `return(ID)`, can only return a code for the token class.

We do not show the details of the code for `install_id`. However, we may suppose that it looks in the symbol table for the lexeme matched by the pattern **id**. Lex makes the lexeme available to routines appearing in the third section through two variables `yytext` and `yyleng`. The variable `yytext` corresponds to the variable that we have been calling *lexeme_beginning*, that is, a pointer to the first character of the lexeme; `yyleng` is an integer telling how long the lexeme is. For example, if `install_id` fails to find the identifier in the symbol table, it might create a new entry for it. The `yyleng` characters of the input, starting at `yytext`, might be copied into a character array and delimited by an end-of-string marker as in Section 2.7. The new symbol-table entry would point to the beginning of this copy.

Numbers are treated similarly by the next rule, and for the last six rules, `yylval` is used to return a code for the particular relational operator found, while the actual return value is the code for token **relop** in each case.

Suppose the lexical analyzer resulting from the program of Fig. 3.18 is given an input consisting of two tabs, the letters `if`, and a blank. The two tabs are the longest initial prefix of the input matched by a pattern, namely the pattern `ws`. The action for `ws` is to do nothing, so the lexical analyzer moves the lexeme-beginning pointer, `yytext`, to the `i` and begins to search for another token.

The next lexeme to be matched is `if`. Note that the patterns `if` and `{id}` both match this lexeme, and no pattern matches a longer string. Since the pattern for keyword `if` precedes the pattern for identifiers in the list of Fig. 3.18, the conflict is resolved in favor of the keyword. In general, this ambiguity-resolving strategy makes it easy to reserve keywords by listing them ahead of the pattern for identifiers.

For another example, suppose <= are the first two characters read. While pattern < matches the first character, it is not the longest pattern matching a prefix of the input. Thus Lex's strategy of selecting the longest prefix matched by a pattern makes it easy to resolve the conflict between < and <= in the expected manner – by choosing <= as the next token. □

### The Lookahead Operator

As we saw in Section 3.1, lexical analyzers for certain programming language constructs need to look ahead beyond the end of a lexeme before they can determine a token with certainty. Recall the example from Fortran of the pair of statements

```
DO 5 I = 1.25
DO 5 I = 1,25
```

In Fortran, blanks are not significant outside of comments and Hollerith

strings, so suppose that all removable blanks are stripped before lexical analysis begins. The above statements then appear to the lexical analyzer as

```
DO5I=1.25
DO5I=1,25
```

In the first statement, we cannot tell until we see the decimal point that the string DO is part of the identifier DO5I. In the second statement, DO is a keyword by itself.

In Lex, we can write a pattern of the form $r_1/r_2$, where $r_1$ and $r_2$ are regular expressions, meaning match a string in $r_1$, but only if followed by a string in $r_2$. The regular expression $r_2$ after the lookahead operator / indicates the right context for a match; it is used only to restrict a match, not to be part of the match. For example, a Lex specification that recognizes the keyword DO in the context above is

```
DO/({letter} | {digit})* = ({letter} | {digit})*,
```

With this specification, the lexical analyzer will look ahead in its input buffer for a sequence of letters and digits followed by an equal sign followed by letters and digits followed by a comma to be sure that it did not have an assignment statement. Then only the characters D and O, preceding the lookahead operator / would be part of the lexeme that was matched. After a successful match, `yytext` points to the D and `yyleng` = 2. Note that this simple lookahead pattern allows DO to be recognized when followed by garbage, like Z4=6Q, but it will never recognize DO that is part of an identifier.

**Example 3.12.** The lookahead operator can be used to cope with another difficult lexical analysis problem in Fortran: distinguishing keywords from identifiers. For example, the input

```
IF(I, J) = 3
```

is a perfectly good Fortran assignment statement, not a logical if-statement. One way to specify the keyword IF using Lex is to define its possible right contexts using the lookahead operator. The simple form of the logical if-statement is

```
IF ( condition ) statement
```

Fortran 77 introduced another form of the logical if-statement:

```
IF ( condition ) THEN
     then_block
ELSE
     else_block
END IF
```

We note that every unlabeled Fortran statement begins with a letter and that every right parenthesis used for subscripting or operand grouping must be followed by an operator symbol such as =, +, or comma, another right

parenthesis, or the end of the statement. Such a right parenthesis cannot be followed by a letter. In this situation, to confirm that `IF` is a keyword rather than an array name we scan forward looking for a right parenthesis followed by a letter before seeing a newline character (we assume continuation cards "cancel" the previous newline character). This pattern for the keyword `IF` can be written as

```
IF / \( .* \) {letter}
```

The dot stands for "any character but newline" and the backslashes in front of the parentheses tell Lex to treat them literally, not as metasymbols for grouping in regular expressions (see Exercise 3.10). □

Another way to attack the problem posed by if-statements in Fortran is, after seeing `IF(`, to determine whether `IF` has been declared an array. We scan for the full pattern indicated above only if it has been so declared. Such tests make the automatic implementation of a lexical analyzer from a Lex specification harder, and they may even cost time in the long run, since frequent checks must be made by the program simulating a transition diagram to determine whether any such tests must be made. It should be noted that tokenizing Fortran is such an irregular task that it is frequently easier to write an ad hoc lexical analyzer for Fortran in a conventional programming language than it is to use an automatic lexical analyzer generator.

## 3.6 FINITE AUTOMATA

A *recognizer* for a language is a program that takes as input a string $x$ and answers "yes" if $x$ is a sentence of the language and "no" otherwise. We compile a regular expression into a recognizer by constructing a generalized transition diagram called a finite automaton. A finite automaton can be deterministic or nondeterministic, where "nondeterministic" means that more than one transition out of a state may be possible on the same input symbol.

Both deterministic and nondeterministic finite automata are capable of recognizing precisely the regular sets. Thus they both can recognize exactly what regular expressions can denote. However, there is a time-space tradeoff; while deterministic finite automata can lead to faster recognizers than nondeterministic automata, a deterministic finite automaton can be much bigger than an equivalent nondeterministic automaton. In the next section, we present methods for converting regular expressions into both kinds of finite automata. The conversion into a nondeterministic automaton is more direct so we discuss these automata first.

The examples in this section and the next deal primarily with the language denoted by the regular expression $(a|b)^*abb$, consisting of the set of all strings of $a$'s and $b$'s ending in $abb$. Similar languages arise in practice. For example, a regular expression for the names of all files that end in `.o` is of the form `(.|o|c)*.o`, with $c$ representing any character other than a dot or an o. As another example, after the opening `/*`, comments in C consist of

any sequence of characters ending in */, with the added requirement that no proper prefix ends in */.

### Nondeterministic Finite Automata

A *nondeterministic finite automaton* (NFA, for short) is a mathematical model that consists of

1. a set of *states* $S$
2. a set of input symbols $\Sigma$ (the *input symbol alphabet*)
3. a transition function *move* that maps state-symbol pairs to sets of states
4. a state $s_0$ that is distinguished as the *start* (or *initial*) *state*
5. a set of states $F$ distinguished as *accepting* (or *final*) *states*

An NFA can be represented diagrammatically by a labeled directed graph, called a *transition graph*, in which the nodes are the states and the labeled edges represent the transition function. This graph looks like a transition diagram, but the same character can label two or more transitions out of one state, and edges can be labeled by the special symbol $\epsilon$ as well as by input symbols.

The transition graph for an NFA that recognizes the language $(a|b)^*abb$ is shown in Fig. 3.19. The set of states of the NFA is {0, 1, 2, 3} and the input symbol alphabet is {$a$, $b$}. State 0 in Fig. 3.19 is distinguished as the start state, and the accepting state 3 is indicated by a double circle.

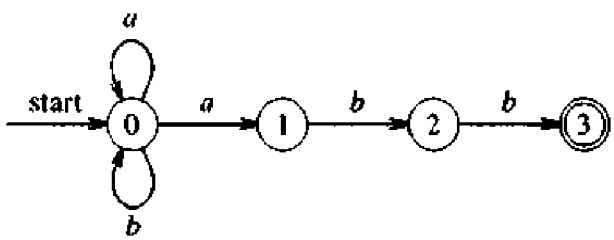



**Fig. 3.19.** A nondeterministic finite automaton.

When describing an NFA, we use the transition graph representation. In a computer, the transition function of an NFA can be implemented in several different ways, as we shall see. The easiest implementation is a *transition table* in which there is a row for each state and a column for each input symbol and $\epsilon$, if necessary. The entry for row $i$ and symbol $a$ in the table is the set of states (or more likely in practice, a pointer to the set of states) that can be reached by a transition from state $i$ on input $a$. The transition table for the NFA of Fig. 3.19 is shown in Fig. 3.20.

The transition table representation has the advantage that it provides fast access to the transitions of a given state on a given character; its disadvantage is that it can take up a lot of space when the input alphabet is large and most transitions are to the empty set. Adjacency list representations of the

| STATE | INPUT SYMBOL | |
|---|---|---|
| | *a* | *b* |
| 0 | {0, 1} | {0} |
| 1 | – | {2} |
| 2 | – | {3} |

**Fig. 3.20.** Transition table for the finite automaton of Fig. 3.19.

transition function provide more compact implementations, but access to a given transition is slower. It should be clear that we can easily convert any one of these implementations of a finite automaton into another.

An NFA *accepts* an input string $x$ if and only if there is some path in the transition graph from the start state to some accepting state, such that the edge labels along this path spell out $x$. The NFA of Fig. 3.19 accepts the input strings *abb*, *aabb*, *babb*, *aaabb*, $\cdots$. For example, *aabb* is accepted by the path from 0, following the edge labeled *a* to state 0 again, then to states 1, 2, and 3 via edges labeled *a*, *b*, and *b*, respectively.

A path can be represented by a sequence of state transitions called *moves*. The following diagram shows the moves made in accepting the input string *aabb*:

$$0 \xrightarrow{a} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{b} 3$$

In general, more than one sequence of moves can lead to an accepting state. Notice that several other sequences of moves may be made on the input string *aabb*, but none of the others happen to end in an accepting state. For example, another sequence of moves on input *aabb* keeps re-entering the non-accepting state 0:

$$0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{b} 0$$

The *language defined by* an NFA is the set of input strings it accepts. It is not hard to show that the NFA of Fig. 3.19 accepts $(a|b)^*abb$.

**Example 3.13.** In Fig. 3.21, we see an NFA to recognize $aa^*|bb^*$. String *aaa* is accepted by moving through states 0, 1, 2, 2, and 2. The labels of these edges are $\epsilon$, *a*, *a*, and *a*, whose concatenation is *aaa*. Note that $\epsilon$'s "disappear" in a concatenation. □

### Deterministic Finite Automata

A *deterministic finite automaton* (DFA, for short) is a special case of a nondeterministic finite automaton in which

1. no state has an $\epsilon$-transition, i.e., a transition on input $\epsilon$, and

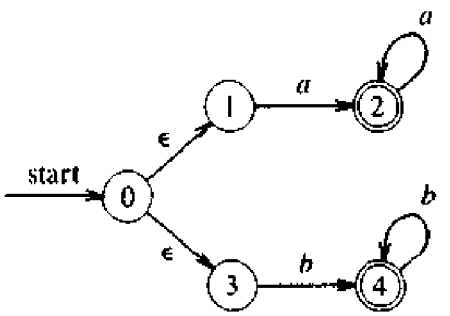



**Fig. 3.21.** NFA accepting $aa^* \mid bb^*$.

2. for each state $s$ and input symbol $a$, there is at most one edge labeled $a$ leaving $s$.

A deterministic finite automaton has at most one transition from each state on any input. If we are using a transition table to represent the transition function of a DFA, then each entry in the transition table is a single state. As a consequence, it is very easy to determine whether a deterministic finite automaton accepts an input string, since there is at most one path from the start state labeled by that string. The following algorithm shows how to simulate the behavior of a DFA on an input string.

**Algorithm 3.1.** Simulating a DFA.

*Input.* An input string $x$ terminated by an end-of-file character **eof**. A DFA $D$ with start state $s_0$ and set of accepting states $F$.

*Output.* The answer "yes" if $D$ accepts $x$; "no" otherwise.

*Method.* Apply the algorithm in Fig. 3.22 to the input string $x$. The function *move*$(s, c)$ gives the state to which there is a transition from state $s$ on input character $c$. The function *nextchar* returns the next character of the input string $x$. □

```
s := s0;
c := nextchar;
while c ≠ eof do
        s := move(s, c);
        c := nextchar
end;
if s is in F then
        return "yes"
else return "no";
```

**Fig. 3.22.** Simulating a DFA.

**Example 3.14.** In Fig. 3.23, we see the transition graph of a deterministic finite automaton accepting the same language $(a|b)^*abb$ as that accepted by the NFA of Fig. 3.19. With this DFA and the input string *ababb*, Algorithm 3.1 follows the sequence of states 0, 1, 2, 1, 2, 3 and returns "yes". □

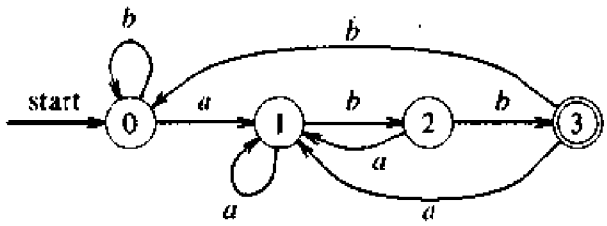



**Fig. 3.23.** DFA accepting $(a|b)^*abb$.

### Conversion of an NFA into a DFA

Note that the NFA of Fig. 3.19 has two transitions from state 0 on input $a$; that is, it may go to state 0 or 1. Similarly, the NFA of Fig. 3.21 has two transitions on $\epsilon$ from state 0. While we have not shown an example of it, a situation where we could choose a transition on $\epsilon$ or on a real input symbol also causes ambiguity. These situations, in which the transition function is multivalued, make it hard to simulate an NFA with a computer program. The definition of acceptance merely asserts that there must be some path labeled by the input string in question leading from the start state to an accepting state. But if there are many paths that spell out the same input string, we may have to consider them all before we find one that leads to acceptance or discover that no path leads to an accepting state.

We now present an algorithm for constructing from an NFA a DFA that recognizes the same language. This algorithm, often called the *subset construction*, is useful for simulating an NFA by a computer program. A closely related algorithm plays a fundamental role in the construction of LR parsers in the next chapter.

In the transition table of an NFA, each entry is a set of states; in the transition table of a DFA, each entry is just a single state. The general idea behind the NFA-to-DFA construction is that each DFA state corresponds to a set of NFA states. The DFA uses its state to keep track of all possible states the NFA can be in after reading each input symbol. That is to say, after reading input $a_1 a_2 \cdots a_n$, the DFA is in a state that represents the subset $T$ of the states of the NFA that are reachable from the NFA's start state along some path labeled $a_1 a_2 \cdots a_n$. The number of states of the DFA can be exponential in the number of states of the NFA, but in practice this worst case occurs rarely.

**Algorithm 3.2.** (*Subset construction.*) Constructing a DFA from an NFA.

*Input.* An NFA $N$.

*Output.* A DFA $D$ accepting the same language.

*Method.* Our algorithm constructs a transition table *Dtran* for $D$. Each DFA state is a set of NFA states and we construct *Dtran* so that $D$ will simulate "in parallel" all possible moves $N$ can make on a given input string.

We use the operations in Fig. 3.24 to keep track of sets of NFA states ($s$ represents an NFA state and $T$ a set of NFA states).

| OPERATION | DESCRIPTION |
|---|---|
| $\epsilon$-*closure*($s$) | Set of NFA states reachable from NFA state $s$ on $\epsilon$-transitions alone. |
| $\epsilon$-*closure*($T$) | Set of NFA states reachable from some NFA state $s$ in $T$ on $\epsilon$-transitions alone. |
| *move*($T$, $a$) | Set of NFA states to which there is a transition on input symbol $a$ from some NFA state $s$ in $T$. |

**Fig. 3.24.** Operations on NFA states.

Before it sees the first input symbol, $N$ can be in any of the states in the set $\epsilon$-*closure*($s_0$), where $s_0$ is the start state of $N$. Suppose that exactly the states in set $T$ are reachable from $s_0$ on a given sequence of input symbols, and let $a$ be the next input symbol. On seeing $a$, $N$ can move to any of the states in the set *move*($T$, $a$). When we allow for $\epsilon$-transitions, $N$ can be in any of the states in $\epsilon$-*closure*(*move*($T$, $a$)), after seeing the $a$.

```
initially, ε-closure(s0) is the only state in Dstates and it is unmarked;
while there is an unmarked state T in Dstates do begin
    mark T;
    for each input symbol a do begin
        U := ε-closure(move(T, a));
        if U is not in Dstates then
            add U as an unmarked state to Dstates;
        Dtran[T, a] := U
    end
end
```

**Fig. 3.25.** The subset construction.

We construct *Dstates*, the set of states of $D$, and *Dtran*, the transition table for $D$, in the following manner. Each state of $D$ corresponds to a set of NFA

states that $N$ could be in after reading some sequence of input symbols including all possible $\epsilon$-transitions before or after symbols are read. The start state of $D$ is $\epsilon$-*closure*($s_0$). States and transitions are added to $D$ using the algorithm of Fig. 3.25. A state of $D$ is an accepting state if it is a set of NFA states containing at least one accepting state of $N$.

```
push all states in T onto stack;
initialize ε-closure(T) to T;
while stack is not empty do begin
        pop t, the top element, off of stack;
        for each state u with an edge from t to u labeled ε do
            if u is not in ε-closure(T) do begin
                add u to ε-closure(T);
                push u onto stack
            end
end
```

**Fig. 3.26.** Computation of $\epsilon$-*closure*.

The computation of $\epsilon$-*closure*($T$) is a typical process of searching a graph for nodes reachable from a given set of nodes. In this case the states of $T$ are the given set of nodes, and the graph consists of just the $\epsilon$-labeled edges of the NFA. A simple algorithm to compute $\epsilon$-*closure*($T$) uses a stack to hold states whose edges have not been checked for $\epsilon$-labeled transitions. Such a procedure is shown in Fig. 3.26. □

**Example 3.15.** Figure 3.27 shows another NFA $N$ accepting the language $(a|b)^*abb$. (It happens to be the one in the next section, which will be mechanically constructed from the regular expression.) Let us apply Algorithm 3.2 to $N$. The start state of the equivalent DFA is $\epsilon$-*closure*(0), which is $A = \{0, 1, 2, 4, 7\}$, since these are exactly the states reachable from state 0 via a path in which every edge is labeled $\epsilon$. Note that a path can have no edges, so 0 is reached from itself by such a path.

The input symbol alphabet here is $\{a, b\}$. The algorithm of Fig. 3.25 tells us to mark $A$ and then to compute

$$\epsilon\text{-}closure(move(A, a)).$$

We first compute $move(A, a)$, the set of states of $N$ having transitions on $a$ from members of $A$. Among the states 0, 1, 2, 4 and 7, only 2 and 7 have such transitions, to 3 and 8, so

$$\epsilon\text{-}closure(move(\{0, 1, 2, 4, 7\}, a)) = \epsilon\text{-}closure(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$$

Let us call this set $B$. Thus, $Dtran[A, a] = B$.

Among the states in $A$, only 4 has a transition on $b$ to 5, so the DFA has a transition on $b$ from $A$ to

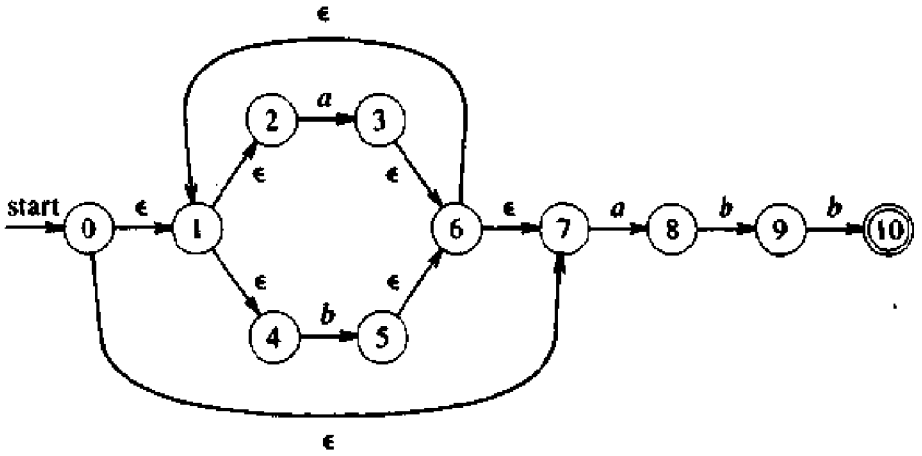



Fig. 3.27. NFA $N$ for $(a|b)^*abb$.

$$C = \epsilon\text{-}closure(\{5\}) = \{1, 2, 4, 5, 6, 7\}$$

Thus, $Dtran[A, b] = C$.

If we continue this process with the now unmarked sets $B$ and $C$, we eventually reach the point where all sets that are states of the DFA are marked. This is certain since there are "only" $2^{11}$ different subsets of a set of eleven states, and a set, once marked, is marked forever. The five different sets of states we actually construct are:

$A = \{0, 1, 2, 4, 7\}$ $\quad D = \{1, 2, 4, 5, 6, 7, 9\}$
$B = \{1, 2, 3, 4, 6, 7, 8\}$ $\quad E = \{1, 2, 4, 5, 6, 7, 10\}$
$C = \{1, 2, 4, 5, 6, 7\}$

State $A$ is the start state, and state $E$ is the only accepting state. The complete transition table *Dtran* is shown in Fig. 3.28.

| State | Input Symbol | |
|---|---|---|
| | $a$ | $b$ |
| $A$ | $B$ | $C$ |
| $B$ | $B$ | $D$ |
| $C$ | $B$ | $C$ |
| $D$ | $B$ | $E$ |
| $E$ | $B$ | $C$ |

Fig. 3.28. Transition table *Dtran* for DFA.

Also, a transition graph for the resulting DFA is shown in Fig. 3.29. It should be noted that the DFA of Fig. 3.23 also accepts $(a|b)^*abb$ and has one

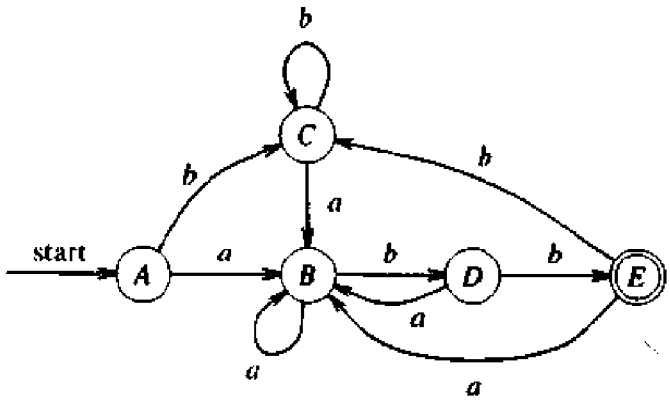



**Fig. 3.29.** Result of applying the subset construction to Fig. 3.27.

fewer state. We discuss the question of minimization of the number of states of a DFA in Section 3.9. □

## 3.7 FROM A REGULAR EXPRESSION TO AN NFA

There are many strategies for building a recognizer from a regular expression, each with its own strengths and weaknesses. One strategy that has been used in a number of text-editing programs is to construct an NFA from a regular expression and then to simulate the behavior of the NFA on an input string using Algorithms 3.3 and 3.4 of this section. If run-time speed is essential, we can convert the NFA into a DFA using the subset construction of the previous section. In Section 3.9, we see an alternative implementation of a DFA from a regular expression in which an intervening NFA is not explicitly constructed. This section concludes with a discussion of time-space tradeoffs in the implementation of recognizers based on NFA and DFA.

### Construction of an NFA from a Regular Expression

We now give an algorithm to construct an NFA from a regular expression. There are many variants of this algorithm, but here we present a simple version that is easy to implement. The algorithm is syntax-directed in that it uses the syntactic structure of the regular expression to guide the construction process. The cases in the algorithm follow the cases in the definition of a regular expression. We first show how to construct automata to recognize $\epsilon$ and any symbol in the alphabet. Then, we show how to construct automata for expressions containing an alternation, concatenation, or Kleene closure operator. For example, for the expression $r|s$, we construct an NFA inductively from the NFA's for $r$ and $s$.

As the construction proceeds, each step introduces at most two new states, so the resulting NFA constructed for a regular expression has at most twice as many states as there are symbols and operators in the regular expression.

**Algorithm 3.3.** (*Thompson's construction.*) An NFA from a regular expression.

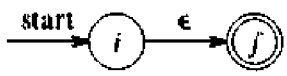
*Input.* A regular expression $r$ over an alphabet $\Sigma$.

*Output.* An NFA $N$ accepting $L(r)$.

*Method.* We first parse $r$ into its constituent subexpressions. Then, using rules (1) and (2) below, we construct NFA's for each of the basic symbols in $r$ (those that are either $\epsilon$ or an alphabet symbol). The basic symbols correspond to parts (1) and (2) in the definition of a regular expression. It is important to understand that if a symbol $a$ occurs several times in $r$, a separate NFA is constructed for each occurrence.

Then, guided by the syntactic structure of the regular expression $r$, we combine these NFA's inductively using rule (3) below until we obtain the NFA for the entire expression. Each intermediate NFA produced during the course of the construction corresponds to a subexpression of $r$ and has several important properties: it has exactly one final state, no edge enters the start state, and no edge leaves the final state.

1. For $\epsilon$, construct the NFA



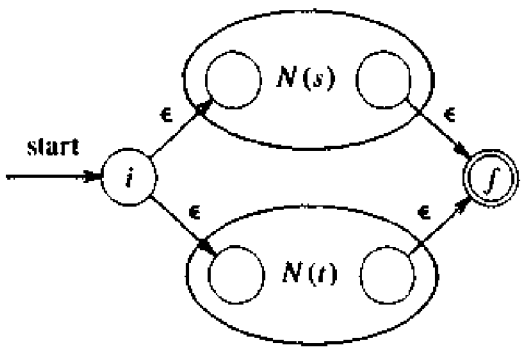

Here $i$ is a new start state and $f$ a new accepting state. Clearly, this NFA recognizes $\{\epsilon\}$.

2. For $a$ in $\Sigma$, construct the NFA


Again $i$ is a new start state and $f$ a new accepting state. This machine recognizes $\{a\}$.

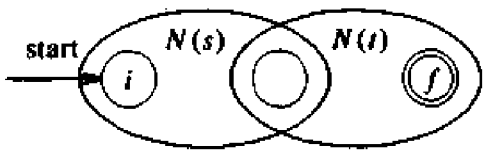
3. Suppose $N(s)$ and $N(t)$ are NFA's for regular expressions $s$ and $t$.

   a) For the regular expression $s|t$, construct the following composite NFA $N(s|t)$:



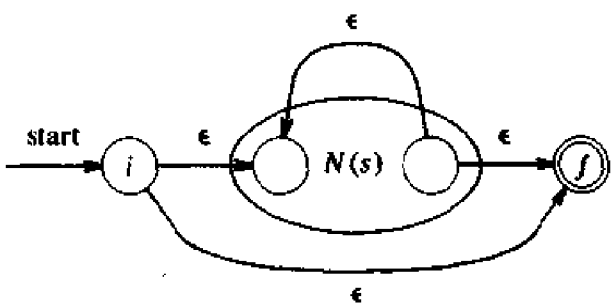
Here $i$ is a new start state and $f$ a new accepting state. There is a transition on $\epsilon$ from $i$ to the start states of $N(s)$ and $N(t)$. There is a transition on $\epsilon$ from the accepting states of $N(s)$ and $N(t)$ to the new accepting state $f$. The start and accepting states of $N(s)$ and $N(t)$ are not start or accepting states of $N(s|t)$. Note that any path from $i$ to $f$ must pass through either $N(s)$ or $N(t)$ exclusively. Thus, we see that the composite NFA recognizes $L(s) \cup L(t)$.

b) For the regular expression $st$, construct the composite NFA $N(st)$:




The start state of $N(s)$ becomes the start state of the composite NFA and the accepting state of $N(t)$ becomes the accepting state of the composite NFA. The accepting state of $N(s)$ is merged with the start state of $N(t)$; that is, all transitions from the start state of $N(t)$ become transitions from the accepting state of $N(s)$. The new merged state loses its status as a start or accepting state in the composite NFA. A path from $i$ to $f$ must go first through $N(s)$ and then through $N(t)$, so the label of that path will be a string in $L(s)L(t)$. Since no edge enters the start state of $N(t)$ or leaves the accepting state of $N(s)$, there can be no path from $i$ to $f$ that travels from $N(t)$ back to $N(s)$. Thus, the composite NFA recognizes $L(s)L(t)$.

c) For the regular expression $s^*$, construct the composite NFA $N(s^*)$:




Here $i$ is a new start state and $f$ a new accepting state. In the composite NFA, we can go from $i$ to $f$ directly, along an edge labeled $\epsilon$, representing the fact that $\epsilon$ is in $(L(s))^*$, or we can go from $i$ to $f$ passing through $N(s)$ one or more times. Clearly, the composite NFA recognizes $(L(s))^*$.

d) For the parenthesized regular expression $(s)$, use $N(s)$ itself as the NFA.

Every time we construct a new state, we give it a distinct name. In this way, no two states of any component NFA can have the same name. Even if the same symbol appears several times in $r$, we create for each instance of that symbol a separate NFA with its own states. □

We can verify that each step of the construction of Algorithm 3.3 produces an NFA that recognizes the correct language. In addition, the construction produces an NFA $N(r)$ with the following properties.

1. $N(r)$ has at most twice as many states as the number of symbols and operators in $r$. This follows from the fact each step of the construction creates at most two new states.
2. $N(r)$ has exactly one start state and one accepting state. The accepting state has no outgoing transitions. This property holds for each of the constituent automata as well.
3. Each state of $N(r)$ has either one outgoing transition on a symbol in $\Sigma$ or at most two outgoing $\epsilon$-transitions.

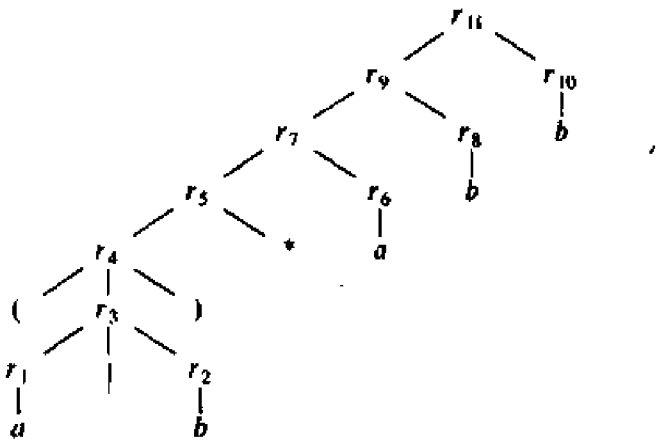
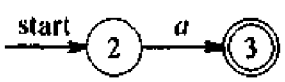
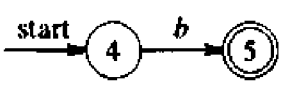



**Fig. 3.30.** Decomposition of $(a|b)^*abb$.

**Example 3.16.** Let us use Algorithm 3.3 to construct $N(r)$ for the regular expression $r = (a|b)^*abb$. Figure 3.30 shows a parse tree for $r$ that is analogous to the parse trees constructed for arithmetic expressions in Section 2.2. For the constituent $r_1$, the first $a$, we construct the NFA

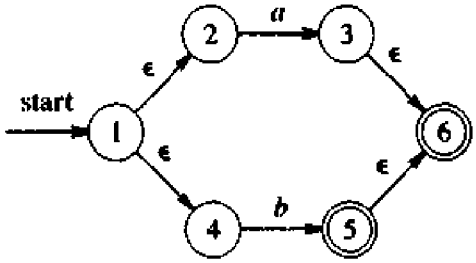



For $r_2$ we construct

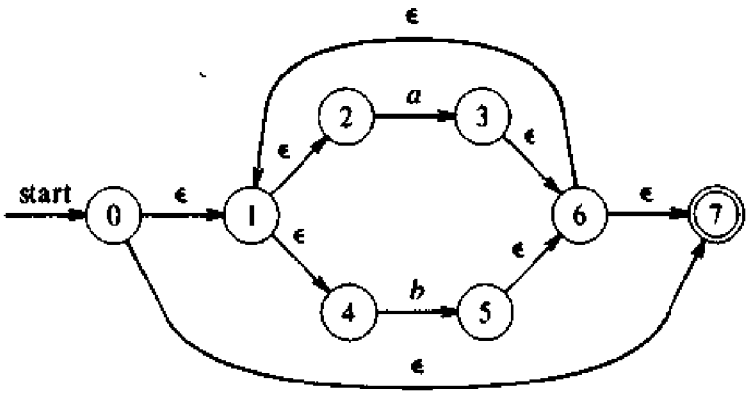



We can now combine $N(r_1)$ and $N(r_2)$ using the union rule to obtain the

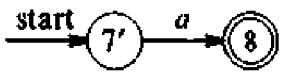
NFA for $r_3 = r_1|r_2$



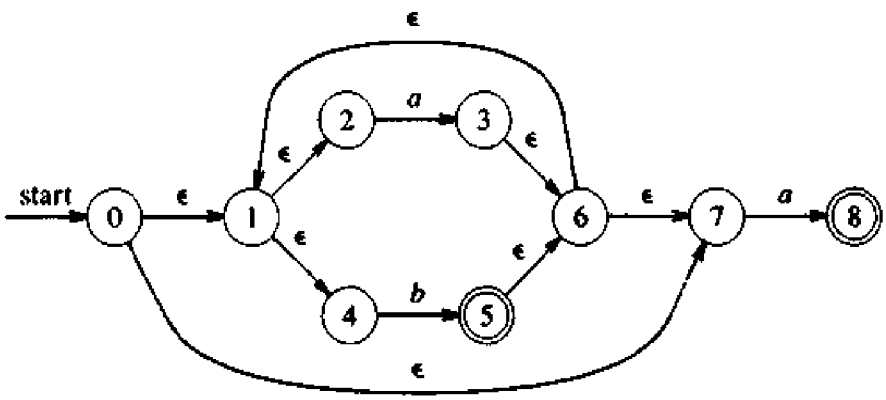

The NFA for $(r_3)$ is the same as that for $r_3$. The NFA for $(r_3)^*$ is then:




The NFA for $r_6 = a$ is




To obtain the automaton for $r_5 r_6$, we merge states 7 and 7', calling the resulting state 7, to obtain




Continuing in this fashion we obtain the NFA for $r_{11} = (a|b)^*abb$ that was first exhibited in Fig. 3.27. □

### Two-Stack Simulation of an NFA

We now present an algorithm that, given an NFA $N$ constructed by Algorithm 3.3 and an input string $x$, determines whether $N$ accepts $x$. The algorithm works by reading the input one character at a time and computing the complete set of states that $N$ could be in after having read each prefix of the input. The algorithm takes advantage of the special properties of the NFA produced by Algorithm 3.3 to compute each set of nondeterministic states efficiently. It can be implemented to run in time proportional to $|N| \times |x|$, where $|N|$ is the number of states in $N$ and $|x|$ is the length of $x$.

**Algorithm 3.4.** Simulating an NFA.

*Input.* An NFA $N$ constructed by Algorithm 3.3 and an input string $x$. We assume $x$ is terminated by an end-of-file character **eof**. $N$ has start state $s_0$ and set of accepting states $F$.

*Output.* The answer "yes" if $N$ accepts $x$; "no" otherwise.

*Method.* Apply the algorithm sketched in Fig. 3.31 to the input string $x$. The algorithm in effect performs the subset construction at run time. It computes a transition from the current set of states $S$ to the next set of states in two stages. First, it determines *move*($S$, $a$), all states that can be reached from a state in $S$ by a transition on $a$, the current input character. Then, it computes the $\epsilon$-*closure* of *move*($S$, $a$), that is, all states that can be reached from *move*($S$, $a$) by zero or more $\epsilon$-transitions. The algorithm uses the function *nextchar* to read the characters of $x$, one at a time. When all characters of $x$ have been seen, the algorithm returns "yes" if an accepting state is in the set $S$ of current states; "no", otherwise. □

```
S := ε-closure({s0});
a := nextchar;
while a ≠ eof do begin
        S := ε-closure(move(S, a));
        a := nextchar
end
if S ∩ F ≠ ∅ then
        return "yes";
else return "no";
```

**Fig. 3.31.** Simulating the NFA of Algorithm 3.3.

Algorithm 3.4 can be efficiently implemented using two stacks and a bit vector indexed by NFA states. We use one stack to keep track of the current set of nondeterministic states and the other stack to compute the next set of nondeterministic states. We can use the algorithm in Fig. 3.26 to compute the $\epsilon$-*closure*. The bit vector can be used to determine in constant time whether a

nondeterministic state is already on a stack so that we do not add it twice. Once we have computed the next state on the second stack, we can interchange the roles of the two stacks. Since each nondeterministic state has at most two out-transitions, each state can give rise to at most two new states in a transition. Let us write $|N|$ for the number of states of $N$. Since there can be at most $|N|$ states on a stack, the computation of the next set of states from the current set of states can be done in time proportional to $|N|$. Thus, the total time needed to simulate the behavior of $N$ on input $x$ is proportional to $|N| \times |x|$.

**Example 3.17.** Let $N$ be the NFA of Fig. 3.27 and let $x$ be the string consisting of the single character $a$. The start state is $\epsilon$-*closure*($\{0\}$) = $\{0, 1, 2, 4, 7\}$. On input symbol $a$ there is a transition from 2 to 3 and from 7 to 8. Thus, $T$ is $\{3, 8\}$. Taking the $\epsilon$-*closure* of $T$ gives us the next state $\{1, 2, 3, 4, 6, 7, 8\}$. Since none of these nondeterministic states is accepting, the algorithm returns "no."

Notice that Algorithm 3.4 does the subset construction at run-time. For example, compare the above transitions with the states of the DFA in Fig. 3.29 constructed from the NFA of Fig. 3.27. The start and next state sets on input $a$ correspond to states A and B of the DFA. □

### Time-Space Tradeoffs

Given a regular expression $r$ and an input string $x$, we now have two methods for determining whether $x$ is in $L(r)$. One approach is to use Algorithm 3.3 to construct an NFA $N$ from $r$. This construction can be done in $O(|r|)$ time, where $|r|$ is the length of $r$. $N$ has at most twice as many states as $|r|$, and at most two transitions from each state, so a transition table for $N$ can be stored in $O(|r|)$ space. We can then use Algorithm 3.4 to determine whether $N$ accepts $x$ in $O(|r| \times |x|)$ time. Thus, using this approach, we can determine whether $x$ is in $L(r)$ in total time proportional to the length of $r$ times the length of $x$. This approach has been used in a number of text editors to search for regular expression patterns when the target string $x$ is generally not very long.

A second approach is to construct a DFA from the regular expression $r$ by applying Thompson's construction to $r$ and then the subset construction, Algorithm 3.2, to the resulting NFA. (An implementation that avoids constructing the intermediate NFA explicitly is given in Section 3.9.) Implementing the transition function with a transition table, we can use Algorithm 3.1 to simulate the DFA on input $x$ in time proportional to the length of $x$, independent of the number of states in the DFA. This approach has often been used in pattern-matching programs that search text files for regular expression patterns. Once the finite automaton has been constructed, the searching can proceed very rapidly, so this approach is advantageous when the target string $x$ is very long.

There are, however, certain regular expressions whose smallest DFA has a

number of states that is exponential in the size of the regular expression. For example, the regular expression $(a|b)^*a(a|b)(a|b)\cdots(a|b)$, where there are $n-1$ $(a|b)$'s at the end, has no DFA with fewer than $2^n$ states. This regular expression denotes any string of $a$'s and $b$'s in which the $n$th character from the right end is an $a$. It is not hard to prove that any DFA for this expression must keep track of the last $n$ characters it sees on the input; otherwise, it may give an erroneous answer. Clearly, at least $2^n$ states are required to keep track of all possible sequences of $n$ $a$'s and $b$'s. Fortunately, expressions such as this do not occur frequently in lexical analysis applications, but there are applications where similar expressions do arise.

A third approach is to use a DFA, but avoid constructing all of the transition table by using a technique called "lazy transition evaluation." Here, transitions are computed at run time but a transition from a given state on a given character is not determined until it is actually needed. The computed transitions are stored in a cache. Each time a transition is about to be made, the cache is consulted. If the transition is not there, it is computed and stored in the cache. If the cache becomes full, we can erase some previously computed transition to make room for the new transition.

Figure 3.32 summarizes the worst-case space and time requirements for determining whether an input string $x$ is in the language denoted by a regular expression $r$ using recognizers constructed from nondeterministic and deterministic finite automata. The "lazy" technique combines the space requirement of the NFA method with the time requirement of the DFA approach. Its space requirement is the size of the regular expression plus the size of the cache; its observed running time is almost as fast as that of a deterministic recognizer. In some applications, the "lazy" technique is considerably faster than the DFA approach, because no time is wasted computing state transitions that are never used.

| AUTOMATON | SPACE | TIME |
|---|---|---|
| NFA | $O(\|r\|)$ | $O(\|r\| \times \|x\|)$ |
| DFA | $O(2^{\|r\|})$ | $O(\|x\|)$ |

**Fig. 3.32.** Space and time taken to recognize regular expressions.

## 3.8 DESIGN OF A LEXICAL ANALYZER GENERATOR

In this section, we consider the design of a software tool that automatically constructs a lexical analyzer from a program in the Lex language. Although we discuss several methods, and none is precisely identical to that used by the UNIX system Lex command, we refer to these programs for constructing lexical analyzers as Lex compilers.

We assume that we have a specification of a lexical analyzer of the form

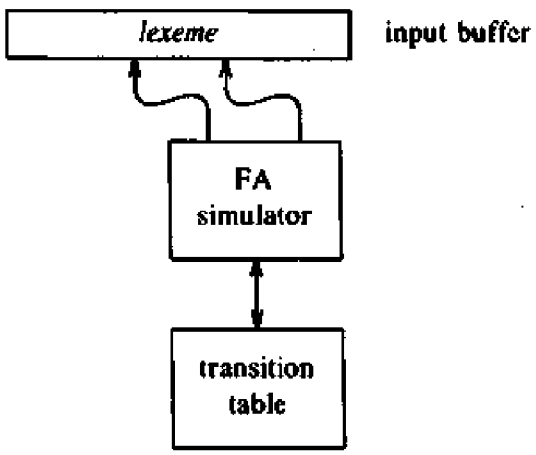
$$
\begin{array}{ll}
p_1 & \{ action_1 \} \\
p_2 & \{ action_2 \} \\
\cdots & \cdots \\
p_n & \{ action_n \}
\end{array}
$$

where, as in Section 3.5, each pattern $p_i$ is a regular expression and each action $action_i$ is a program fragment that is to be executed whenever a lexeme matched by $p_i$ is found in the input.

Our problem is to construct a recognizer that looks for lexemes in the input buffer. If more than one pattern matches, the recognizer is to choose the longest lexeme matched. If there are two or more patterns that match the longest lexeme, the first-listed matching pattern is chosen.

A finite automaton is a natural model around which to build a lexical analyzer, and the one constructed by our Lex compiler has the form shown in Fig. 3.33(b). There is an input buffer with two pointers to it, a lexeme-beginning and a forward pointer, as discussed in Section 3.2. The Lex compiler constructs a transition table for a finite automaton from the regular expression patterns in the Lex specification. The lexical analyzer itself consists of a finite automaton simulator that uses this transition table to look for the regular expression patterns in the input buffer.


(a) Lex compiler.




(b) Schematic lexical analyzer.

**Fig. 3.33.** Model of Lex compiler.

The remainder of this section shows that the implementation of a Lex

compiler can be based on either nondeterministic or deterministic automata. At the end of the last section we saw that the transition table of an NFA for a regular expression pattern can be considerably smaller than that of a DFA, but the DFA has the decided advantage of being able to recognize patterns faster than the NFA.

**Pattern Matching Based on NFA's**

One method is to construct the transition table of a nondeterministic finite automaton $N$ for the composite pattern $p_1 | p_2 | \cdots | p_n$. This can be done by first creating an NFA $N(p_i)$ for each pattern $p_i$ using Algorithm 3.3, then adding a new start state $s_0$, and finally linking $s_0$ to the start state of each $N(p_i)$ with an $\epsilon$-transition, as shown in Fig. 3.34.

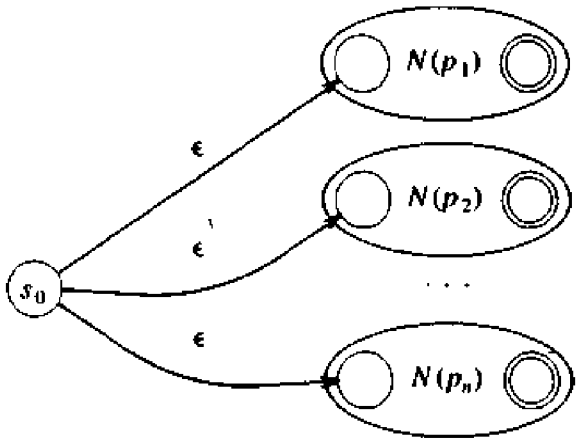



**Fig. 3.34.** NFA constructed from Lex specification.

To simulate this NFA we can use a modification of Algorithm 3.4. The modification ensures that the combined NFA recognizes the longest prefix of the input that is matched by a pattern. In the combined NFA, there is an accepting state for each pattern $p_i$. When we simulate the NFA using Algorithm 3.4, we construct the sequence of sets of states that the combined NFA can be in after seeing each input character. Even if we find a set of states that contains an accepting state, to find the longest match we must continue to simulate the NFA until it reaches *termination*, that is, a set of states from which there are no transitions on the current input symbol.

We presume that the Lex specification is designed so that a valid source program cannot entirely fill the input buffer without having the NFA reach termination. For example, each compiler puts some restriction on the length of an identifier, and violations of this limit will be detected when the input buffer overflows, if not sooner.

To find the correct match, we make two modifications to Algorithm 3.4. First, whenever we add an accepting state to the current set of states, we

record the current input position and the pattern $p_i$ corresponding to this accepting state. If the current set of states already contains an accepting state, then only the pattern that appears first in the Lex specification is recorded. Second, we continue making transitions until we reach termination. Upon termination, we retract the forward pointer to the position at which the last match occurred. The pattern making this match identifies the token found, and the lexeme matched is the string between the lexeme-beginning and forward pointers.

Usually, the Lex specification is such that some pattern, possibly an error pattern, will always match. If no pattern matches, however, we have an error condition for which no provision was made, and the lexical analyzer should transfer control to some default error recovery routine.

**Example 3.18.** A simple example illustrates the above ideas. Suppose we have the following Lex program consisting of three regular expressions and no regular definitions.

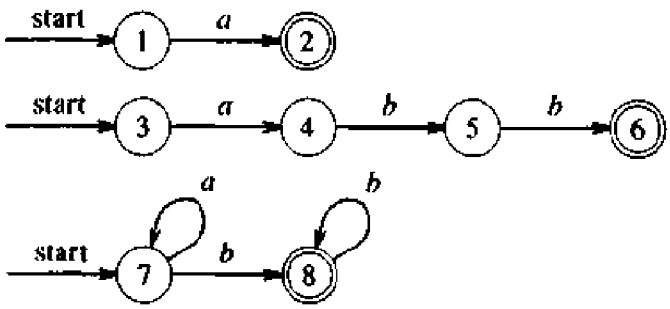
```
a       { } /* actions are omitted here */
abb     { }
a*b+    { }
```

The three tokens above are recognized by the automata of Fig. 3.35(a). We have simplified the third automaton somewhat from what would be produced by Algorithm 3.3. As indicated above, we can convert the NFA's of Fig. 3.35(a) into one combined NFA $N$ shown in 3.35(b).

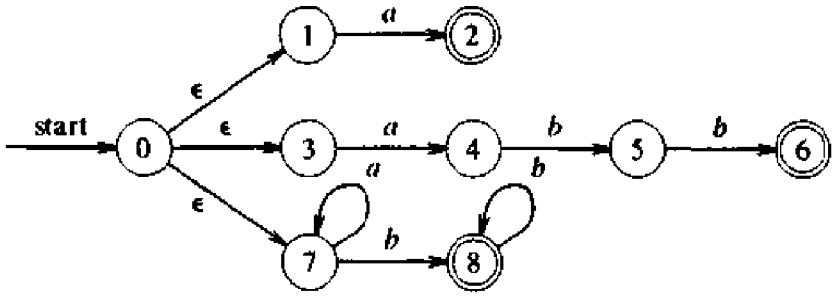
Let us now consider the behavior of $N$ on the input string *aaba* using our modification of Algorithm 3.4. Figure 3.36 shows the sets of states and patterns that match as each character of the input *aaba* is processed. This figure shows that the initial set of states is {0, 1, 3, 7}. States 1, 3, and 7 each have a transition on $a$, to states 2, 4, and 7, respectively. Since state 2 is the accepting state for the first pattern, we record the fact that the first pattern matches after reading the first $a$.

However, there is a transition from state 7 to state 7 on the second input character, so we must continue making transitions. There is a transition from state 7 to state 8 on the input character $b$. State 8 is the accepting state for the third pattern. Once we reach state 8, there are no transitions possible on the next input character $a$ so we have reached termination. Since the last match occurred after we read the third input character, we report that the third pattern has matched the lexeme *aab*. □

The role of $action_i$ associated with the pattern $p_i$ in the Lex specification is as follows. When an instance of $p_i$ is recognized, the lexical analyzer executes the associated program $action_i$. Note that $action_i$ is not executed just because the NFA enters a state that includes the accepting state for $p_i$; $action_i$ is only executed if $p_i$ turns out to be the pattern yielding the longest match.




(a) NFA for $a$, $abb$, and $a^*b^+$.




(b) Combined NFA.

**Fig. 3.35.** NFA recognizing three different patterns.

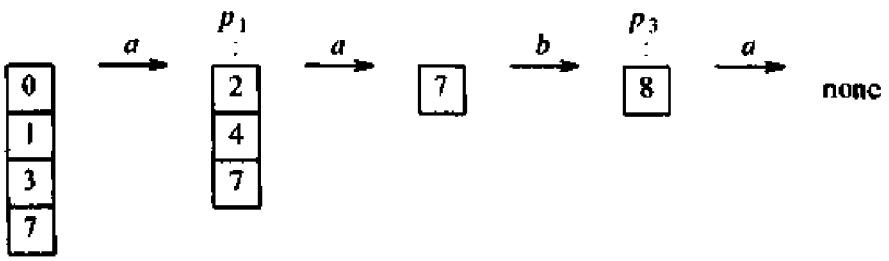



**Fig. 3.36.** Sequence of sets of states entered in processing input *aaba*.

### DFA for Lexical Analyzers

Another approach to the construction of a lexical analyzer from a Lex specification is to use a DFA to perform the pattern matching. The only nuance is to make sure we find the proper pattern matches. The situation is completely analogous to the modified simulation of an NFA just described. When we convert an NFA to a DFA using the subset construction Algorithm 3.2, there

may be several accepting states in a given subset of nondeterministic states. In such a situation, the accepting state corresponding to the pattern listed first in the Lex specification has priority. As in the NFA simulation, the only other modification we need to perform is to continue making state transitions until we reach a state with no next state (i.e., the state $\emptyset$) for the current input symbol. To find the lexeme matched, we return to the last input position at which the DFA entered an accepting state.

| STATE | INPUT SYMBOL | | PATTERN ANNOUNCED |
|---|---|---|---|
| | *a* | *b* | |
| 0137 | 247 | 8 | none |
| 247 | 7 | 58 | *a* |
| 8 | - | 8 | $a^*b^+$ |
| 7 | 7 | 8 | none |
| 58 | - | 68 | $a^*b^+$ |
| 68 | - | 8 | *abb* |

**Fig. 3.37.** Transition table for a DFA.

**Example 3.19.** If we convert the NFA in Figure 3.35 to a DFA, we obtain the transition table in Fig. 3.37, where the states of the DFA have been named by lists of the states of the NFA. The last column in Fig. 3.37 indicates one of the patterns recognized upon entering that DFA state. For example, among NFA states 2, 4, and 7, only 2 is accepting, and it is the accepting state of the automaton for regular expression *a* in Fig. 3.35(a). Thus, DFA state 247 recognizes pattern *a*.

Note that the string *abb* is matched by two patterns, *abb* and $a^*b^+$, recognized at NFA states 6 and 8. DFA state 68, in the last line of the transition table, therefore includes two accepting states of the NFA. We note that *abb* appears before $a^*b^+$ in the translation rules of our Lex specification, so we announce that *abb* has been found in DFA state 68.

On input string *aaba*, the DFA enters the states suggested by the NFA simulation shown in Fig. 3.36. Consider a second example, the input string *aba*. The DFA of Fig. 3.37 starts off in state 0137. On input *a* it goes to state 247. Then on input *b* it progresses to state 58, and on input *a* it has no next state. We have thus reached termination, progressing through the DFA states 0137, then 247, then 58. The last of these includes the accepting NFA state 8 from Fig. 3.35(a). Thus in state 58 the DFA announces that the pattern $a^*b^+$ has been recognized, and selects *ab*, the prefix of the input that led to state 58, as the lexeme. □

### Implementing the Lookahead Operator

Recall from Section 3.4 that the lookahead operator / is necessary in some situations, since the pattern that denotes a particular token may need to describe some trailing context for the actual lexeme. When converting a pattern with / to an NFA, we can treat the / as if it were $\epsilon$, so that we do not actually look for / on the input. However, if a string denoted by this regular expression is recognized in the input buffer, the end of the lexeme is not the position of the NFA's accepting state. Rather it is at the last occurrence of the state of this NFA having a transition on the (imaginary) /.

**Example 3.20.** The NFA recognizing the pattern for `IF` given in Example 3.12 is shown in Fig. 3.38. State 6 indicates the presence of keyword `IF`; however, we find the token `IF` by scanning backwards to the last occurrence of state 2. □

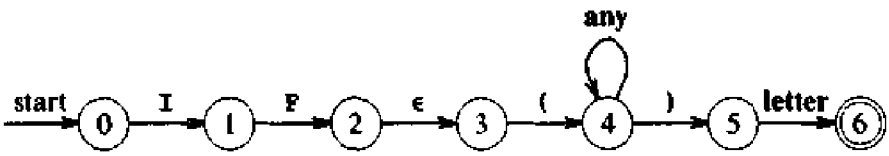



Fig. 3.38. NFA recognizing Fortran keyword `IF`.

## 3.9 OPTIMIZATION OF DFA-BASED PATTERN MATCHERS

In this section, we present three algorithms that have been used to implement and optimize pattern matchers constructed from regular expressions. The first algorithm is suitable for inclusion in a Lex compiler because it constructs a DFA directly from a regular expression, without constructing an intermediate NFA along the way.

The second algorithm minimizes the number of states of any DFA, so it can be used to reduce the size of a DFA-based pattern matcher. The algorithm is efficient; its running time is $O(n \log n)$, where $n$ is the number of states in the DFA. The third algorithm can be used to produce fast but more compact representations for the transition table of a DFA than a straightforward two-dimensional table.

### Important States of an NFA

Let us call a state of an NFA *important* if it has a non-$\epsilon$ out-transition. The subset construction in Fig. 3.25 uses only the important states in a subset $T$ when it determines $\epsilon$-*closure*(*move*($T$, $a$)), the set of states that is reachable from $T$ on input $a$. The set *move*($s$, $a$) is nonempty only if state $s$ is important. During the construction, two subsets can be identified if they have the same important states, and either both or neither include accepting states of the NFA.

When the subset construction is applied to an NFA obtained from a regular expression by Algorithm 3.3, we can exploit the special properties of the NFA to combine the two constructions. The combined construction relates the important states of the NFA with the symbols in the regular expression. Thompson's construction builds an important state exactly when a symbol in the alphabet appears in a regular expression. For example, important states will be constructed for each $a$ and $b$ in $(a|b)^*abb$.

Moreover, the resulting NFA has exactly one accepting state, but the accepting state is not important because it has no transitions leaving it. By concatenating a unique right-end marker # to a regular expression $r$, we give the accepting state of $r$ a transition on #, making it an important state of the NFA for $r\#$. In other words, by using the augmented regular expression $(r)\#$ we can forget about accepting states as the subset construction proceeds; when the construction is complete, any DFA state with a transition on # must be an accepting state.

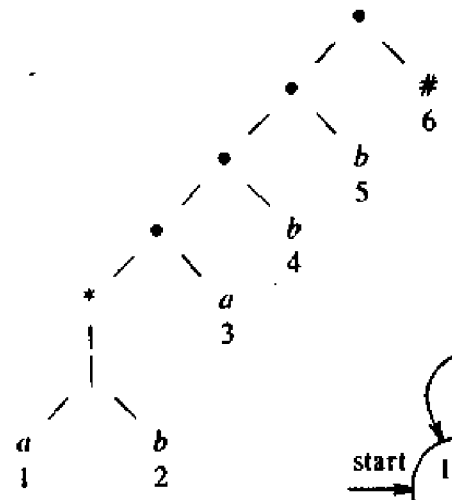
We represent an augmented regular expression by a syntax tree with basic symbols at the leaves and operators at the interior nodes. We refer to an interior node as a *cat-node*, *or-node*, or *star-node* if it is labeled by a concatenation, |, or * operator, respectively. Figure 3.39(a) shows a syntax tree for an augmented regular expression with cat-nodes marked by dots. The syntax tree for a regular expression can be constructed in the same manner as a syntax tree for an arithmetic expression (see Chapter 2).

Leaves in the syntax tree for a regular expression are labeled by alphabet symbols or by $\epsilon$. To each leaf not labeled by $\epsilon$ we attach a unique integer and refer to this integer as the *position* of the leaf and also as a position of its symbol. A repeated symbol therefore has several positions. Positions are shown below the symbols in the syntax tree of Fig. 3.39(a). The numbered states in the NFA of Fig. 3.39(c) correspond to the positions of the leaves in the syntax tree in Fig. 3.39(a). It is no coincidence that these states are the important states of the NFA. Non-important states are named by upper case letters in Fig. 3.39(c).

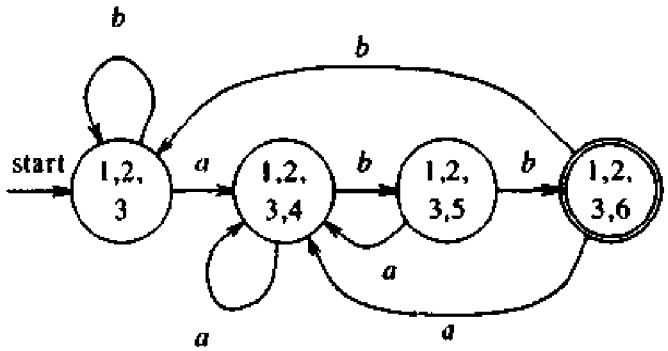
The DFA in Fig. 3.39(b) can be obtained from the NFA in Fig. 3.39(c) if we apply the subset construction and identify subsets containing the same important states. The identification results in one fewer state being constructed, as a comparison with Fig. 3.29 shows.

### From a Regular Expression to a DFA

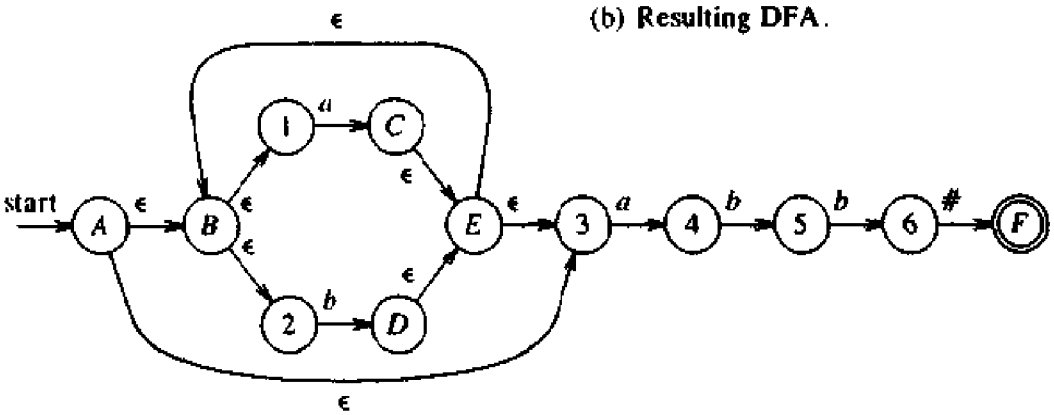
In this section, we show how to construct a DFA directly from an augmented regular expression $(r)\#$. We begin by constructing a syntax tree $T$ for $(r)\#$ and then computing four functions: *nullable*, *firstpos*, *lastpos*, and *followpos*, by making traversals over $T$. Finally, we construct the DFA from *followpos*. The functions *nullable*, *firstpos*, and *lastpos* are defined on the nodes of the syntax tree and are used to compute *followpos*, which is defined on the set of positions.




(a) Syntax tree for $(a|b)^*abb\#$.




(b) Resulting DFA.




(c) Underlying NFA.

**Fig. 3.39.** DFA and NFA constructed from $(a|b)^*abb\#$.

Remembering the equivalence between the important NFA states and the positions of the leaves in the syntax tree of the regular expression, we can short-circuit the construction of the NFA by building the DFA whose states correspond to sets of positions in the tree. The $\epsilon$-transitions of the NFA represent some fairly complicated structure of the positions; in particular, they encode the information regarding when one position can follow another. That is, each symbol in an input string to a DFA can be matched by certain positions. An input symbol $c$ can only be matched by positions at which there is a $c$, but not every position with a $c$ can necessarily match a particular occurrence of $c$ in the input stream.

The notion of a position matching an input symbol will be defined in terms

of the function *followpos* on positions of the syntax tree. If $i$ is a position, then *followpos*($i$) is the set of positions $j$ such that there is some input string $\cdots cd \cdots$ such that $i$ corresponds to this occurrence of $c$ and $j$ to this occurrence of $d$.

**Example 3.21.** In Fig. 3.39(a), *followpos*(1) = {1, 2, 3}. The reasoning is that if we see an $a$ corresponding to position 1, then we have just seen an occurrence of $a|b$ in the closure $(a|b)^*$. We could next see the first position of another occurrence of $a|b$, which explains why 1 and 2 are in *followpos*(1). We could also next see the first position of what follows $(a|b)^*$, that is, position 3. □

In order to compute the function *followpos*, we need to know what positions can match the first or last symbol of a string generated by a given subexpression of a regular expression. (Such information was used informally in Example 3.21.) If $r^*$ is such a subexpression, then every position that can be first in $r$ follows every position that can be last in $r$. Similarly, if $rs$ is a subexpression, then every first position of $s$ follows every last position of $r$.

At each node $n$ of the syntax tree of a regular expression, we define a function *firstpos*($n$) that gives the set of positions that can match the first symbol of a string generated by the subexpression rooted at $n$. Likewise, we define a function *lastpos*($n$) that gives the set of positions that can match the last symbol in such a string. For example, if $n$ is the root of the whole tree in Fig. 3.39(a), then *firstpos*($n$) = {1, 2, 3} and *lastpos*($n$) = {6}. We give an algorithm for computing these functions momentarily.

In order to compute *firstpos* and *lastpos*, we need to know which nodes are the roots of subexpressions that generate languages that include the empty string. Such nodes are called *nullable*, and we define *nullable*($n$) to be true if node $n$ is nullable, false otherwise.

We can now give the rules to compute the functions *nullable*, *firstpos*, *lastpos*, and *followpos*. For the first three functions, we have a basis rule that tells about expressions of a basic symbol, and then three inductive rules that allow us to determine the value of the functions working up the syntax tree from the bottom; in each case the inductive rules correspond to the three operators, union, concatenation, and closure. The rules for *nullable* and *firstpos* are given in Fig. 3.40. The rules for *lastpos*($n$) are the same as those for *firstpos*($n$), but with $c_1$ and $c_2$ reversed, and are not shown.

The first rule for *nullable* states that if $n$ is a leaf labeled $\epsilon$, then *nullable*($n$) is surely true. The second rule states that if $n$ is a leaf labeled by an alphabet symbol, then *nullable*($n$) is false. In this case, each leaf corresponds to a single input symbol, and therefore cannot generate $\epsilon$. The last rule for *nullable* states that if $n$ is a star-node with child $c_1$, then *nullable*($n$) is true, because the closure of an expression generates a language that includes $\epsilon$.

As another example, the fourth rule for *firstpos* states that if $n$ is a cat-node with left child $c_1$ and right child $c_2$, and if *nullable*($c_1$) is true, then

$$\mathit{firstpos}(n) = \mathit{firstpos}(c_1) \cup \mathit{firstpos}(c_2)$$

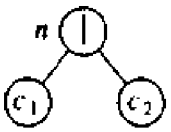
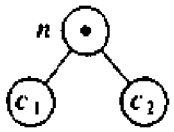
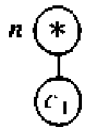
| NODE $n$ | *nullable* ($n$) | *firstpos* ($n$) |
|---|---|---|
| $n$ is a leaf labeled $\epsilon$ | **true** | $\emptyset$ |
| $n$ is a leaf labeled with position $i$ | **false** | $\{i\}$ |
|  | *nullable* ($c_1$) **or** *nullable* ($c_2$) | *firstpos* ($c_1$) $\cup$ *firstpos* ($c_2$) |
|  | *nullable* ($c_1$) **and** *nullable* ($c_2$) | **if** *nullable* ($c_1$) **then** *firstpos* ($c_1$) $\cup$ *firstpos* ($c_2$) **else** *firstpos* ($c_1$) |
|  | **true** | *firstpos* ($c_1$) |

**Fig. 3.40.** Rules for computing *nullable* and *firstpos*.

otherwise, *firstpos* ($n$) = *firstpos* ($c_1$). What this rule says is that if in an expression $rs$, $r$ generates $\epsilon$, then the first positions of $s$ "show through" $r$ and are also first positions of $rs$; otherwise, only the first positions of $r$ are first positions of $rs$. The reasoning behind the remaining rules for *nullable* and *firstpos* are similar.

The function *followpos* ($i$) tells us what positions can follow position $i$ in the syntax tree. Two rules define all the ways one position can follow another.

1. If $n$ is a cat-node with left child $c_1$ and right child $c_2$, and $i$ is a position in *lastpos* ($c_1$), then all positions in *firstpos* ($c_2$) are in *followpos* ($i$).
2. If $n$ is a star-node, and $i$ is a position in *lastpos* ($n$), then all positions in *firstpos* ($n$) are in *followpos* ($i$).

If *firstpos* and *lastpos* have been computed for each node, *followpos* of each position can be computed by making one depth-first traversal of the syntax tree.

**Example 3.22.** Figure 3.41 shows the values of *firstpos* and *lastpos* at all nodes in the tree of Fig. 3.39(a); *firstpos* ($n$) appears to the left of node $n$ and *lastpos* ($n$) to the right. For example, *firstpos* at the leftmost leaf labeled $a$ is $\{1\}$, since this leaf is labeled with position 1. Similarly, *firstpos* of the second leaf is $\{2\}$, since this leaf is labeled with position 2. By the third rule in Fig. 3.40, *firstpos* of their parent is $\{1, 2\}$.

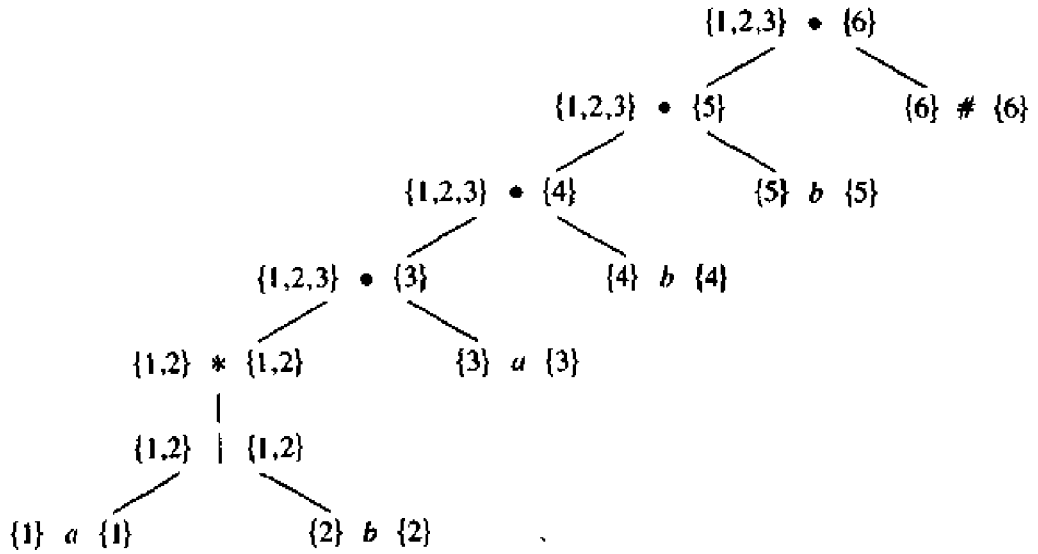



**Fig. 3.41.** *firstpos* and *lastpos* for nodes in syntax tree for $(a|b)^*abb\#$.

The node labeled * is the only nullable node. Thus, by the if-condition of the fourth rule, *firstpos* for the parent of this node (the one representing expression $(a|b)^*a$) is the union of {1, 2} and {3}, which are the *firstpos*'s of its left and right children. On the other hand, the else-condition applies for *lastpos* of this node, since the leaf at position 3 is not nullable. Thus, the parent of the star-node has *lastpos* containing only 3.

Let us now compute *followpos* bottom up for each node of the syntax tree of Fig. 3.41. At the star-node, we add both 1 and 2 to *followpos*(1) and to *followpos*(2) using rule (2). At the parent of the star-node, we add 3 to *followpos*(1) and *followpos*(2) using rule (1). At the next cat-node, we add 4 to *followpos*(3) using rule (1). At the next two cat-nodes we add 5 to *followpos*(4) and 6 to *followpos*(5) using the same rule. This completes the construction of *followpos*. Figure 3.42 summarizes *followpos*.

| NODE | *followpos* |
|---|---|
| 1 | {1, 2, 3} |
| 2 | {1, 2, 3} |
| 3 | {4} |
| 4 | {5} |
| 5 | {6} |
| 6 | - |

**Fig. 3.42.** The function *followpos*.

We can illustrate the function *followpos* by creating a directed graph with a node for each position and directed edge from node $i$ to node $j$ if $j$ is in *followpos*($i$). Figure 3.43 shows this directed graph for *followpos* of Fig.

3.42.

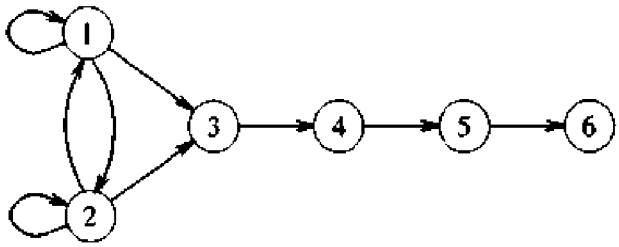



**Fig. 3.43.** Directed graph for the function *followpos*.

It is interesting to note that this diagram would become an NFA without $\epsilon$-transitions for the regular expression in question if we:

1. make all positions in *firstpos* of the root be start states,
2. label each directed edge $(i, j)$ by the symbol at position $j$, and
3. make the position associated with # be the only accepting state.

It should therefore come as no surprise that we can convert the *followpos* graph into a DFA using the subset construction. The entire construction can be carried out on the positions, using the following algorithm. □

**Algorithm 3.5.** Construction of a DFA from a regular expression $r$.

*Input.* A regular expression $r$.

*Output.* A DFA $D$ that recognizes $L(r)$.

*Method.*

1. Construct a syntax tree for the augmented regular expression $(r)$#, where # is a unique endmarker appended to $(r)$.
2. Construct the functions *nullable*, *firstpos*, *lastpos*, and *followpos* by making depth-first traversals of $T$.
3. Construct *Dstates*, the set of states of $D$, and *Dtran*, the transition table for $D$ by the procedure in Fig. 3.44. The states in *Dstates* are sets of positions; initially, each state is "unmarked," and a state becomes "marked" just before we consider its out-transitions. The start state of $D$ is *firstpos*(*root*), and the accepting states are all those containing the position associated with the endmarker #. □

**Example 3.23.** Let us construct a DFA for the regular expression $(a|b)^*abb$. The syntax tree for $((a|b)^*abb)$# is shown in Fig. 3.39(a). *nullable* is true only for the node labeled *. The functions *firstpos* and *lastpos* are shown in Fig. 3.41, and *followpos* is shown in Fig. 3.42.

From Fig. 3.41, *firstpos* of the root is {1, 2, 3}. Let this set be $A$ and

```
initially, the only unmarked state in Dstates is firstpos(root),
    where root is the root of the syntax tree for (r)#;
while there is an unmarked state T in Dstates do begin
    mark T;
    for each input symbol a do begin
        let U be the set of positions that are in followpos(p)
            for some position p in T,
            such that the symbol at position p is a;
        if U is not empty and is not in Dstates then
            add U as an unmarked state to Dstates;
        Dtran[T, a] := U
    end
end
```

**Fig. 3.44.** Construction of DFA.

consider input symbol $a$. Positions 1 and 3 are for $a$, so let $B = followpos(1) \cup followpos(3) = \{1, 2, 3, 4\}$. Since this set has not yet been seen, we set $Dtran[A, a] := B$.

When we consider input $b$, we note that of the positions in $A$, only 2 is associated with $b$, so we must consider the set $followpos(2) = \{1, 2, 3\}$. Since this set has already been seen, we do not add it to *Dstates*, but we add the transition $Dtran[A, b] := A$.

We now continue with $B = \{1, 2, 3, 4\}$. The states and transitions we finally obtain are the same as those that were shown in Fig. 3.39(b). □

### Minimizing the Number of States of a DFA

An important theoretical result is that every regular set is recognized by a minimum-state DFA that is unique up to state names. In this section, we show how to construct this minimum-state DFA by reducing the number of states in a given DFA to the bare minimum without affecting the language that is being recognized. Suppose that we have a DFA $M$ with set of states $S$ and input symbol alphabet $\Sigma$. We assume that every state has a transition on every input symbol.. If that were not the case, we can introduce a new "dead state" $d$, with transitions from $d$ to $d$ on all inputs, and add a transition from state $s$ to $d$ on input $a$ if there was no transition from $s$ on $a$.

We say that string $w$ *distinguishes* state $s$ from state $t$ if, by starting with the DFA $M$ in state $s$ and feeding it input $w$, we end up in an accepting state, but starting in state $t$ and feeding it input $w$, we end up in a nonaccepting state, or vice versa. For example, $\epsilon$ distinguishes any accepting state from any nonaccepting state, and in the DFA of Fig. 3.29, states $A$ and $B$ are distinguished by the input $bb$, since $A$ goes to the nonaccepting state $C$ on input $bb$, while $B$ goes to the accepting state $E$ on that same input.

Our algorithm for minimizing the number of states of a DFA works by finding all groups of states that can be distinguished by some input string. Each group of states that cannot be distinguished is then merged into a single state. The algorithm works by maintaining and refining a partition of the set of states. Each group of states within the partition consists of states that have not yet been distinguished from one another, and any pair of states chosen from different groups have been found distinguishable by some input.

Initially, the partition consists of two groups: the accepting states and the nonaccepting states. The fundamental step is to take some group of states, say $A = \{s_1, s_2, \ldots, s_k\}$ and some input symbol $a$, and look at what transitions states $s_1, \ldots, s_k$ have on input $a$. If these transitions are to states that fall into two or more different groups of the current partition, then we must split $A$ so that the transitions from the subsets of $A$ are all confined to a single group of the current partition. Suppose, for example, that $s_1$ and $s_2$ go to states $t_1$ and $t_2$ on input $a$, and $t_1$ and $t_2$ are in different groups of the partition. Then we must split $A$ into at least two subsets so that one subset contains $s_1$ and the other $s_2$. Note that $t_1$ and $t_2$ are distinguished by some string $w$, so $s_1$ and $s_2$ are distinguished by string $aw$.

We repeat this process of splitting groups in the current partition until no more groups need to be split. While we have justified why states that have been split into different groups really can be distinguished, we have not indicated why states that are not split into different groups are certain not to be distinguishable by any input string. Such is the case, however, and we leave a proof of that fact to the reader interested in the theory (see, for example, Hopcroft and Ullman [1979]). Also left to the interested reader is a proof that the DFA constructed by taking one state for each group of the final partition and then throwing away the dead state and states not reachable from the start state has as few states as any DFA accepting the same language.

**Algorithm 3.6.** Minimizing the number of states of a DFA.

*Input.* A DFA $M$ with set of states $S$, set of inputs $\Sigma$, transitions defined for all states and inputs, start state $s_0$, and set of accepting states $F$.

*Output.* A DFA $M'$ accepting the same language as $M$ and having as few states as possible.

*Method.*

1. Construct an initial partition $\Pi$ of the set of states with two groups: the accepting states $F$ and the nonaccepting states $S - F$.
2. Apply the procedure of Fig. 3.45 to $\Pi$ to construct a new partition $\Pi_{new}$.
3. If $\Pi_{new} = \Pi$, let $\Pi_{final} = \Pi$ and continue with step (4). Otherwise, repeat step (2) with $\Pi := \Pi_{new}$.
4. Choose one state in each group of the partition $\Pi_{final}$ as the *representative*